

Prueba Técnica: Senior QA Engineer

Cloud, Automation & Mobile · LogiTrack

Formato	Respuestas, diagramas, código y capturas
Repositorio	github.com/restrej/prueba-tecnica-cloud-automation-and-mobile
Stack	Python + pytest · Playwright · Appium · k6 · OWASP ZAP · GitHub Actions
Fecha	2026-10-01

Contenido

1. Parte 1. Estrategia de calidad y factibilidad (20 pts)
2. Parte 2. Diseño de casos de prueba (15 pts)
3. Parte 3. Automatización y CI/CD (15 pts)
4. Parte 4. Automatización técnica (25 pts)
5. Parte 5. Rendimiento y seguridad (10 pts)
6. Parte 6. Cloud y troubleshooting (10 pts)
7. Parte 7. Comunicación (5 pts)
8. Bonus. Arquitectura y riesgos (20 pts)
9. Anexo A. Implementación y cómo ejecutarla
10. Anexo B. Evidencias de ejecución

Todo el código citado está en el repositorio y se ejecutó localmente: 148 pruebas automatizadas en verde, prueba de carga k6 de 5 minutos, escaneo DAST con OWASP ZAP y SAST con Bandit. La guía [docs/PASO_A_PASO.md](#) explica cómo reproducir cada resultado.

Parte 1. Estrategia de calidad y factibilidad de automatización

1.1 Estrategia de pruebas para el flujo principal

Flujo: *Pedido recibido* → *orders-api* → *Pub/Sub* → *inventory-service (reserva stock)* → *asignación a operador (PickApp)* → *escaneo* → *shipping-service (guía)* → *notifications-service* → *entrega*.

Principio rector: **pirámide de pruebas** (muchas pruebas rápidas y baratas abajo, pocas lentas y costosas arriba) y **shift-left** (detectar el defecto lo antes posible, donde es más barato corregirlo). Los 4 problemas actuales se asignan explícitamente al nivel que mejor los detecta.

Nivel	Qué valida	Herramientas	Riesgos que cubre
Unit Tests	Reglas de negocio aisladas: asignación (roles, almacén, estado del pedido, idempotencia), cálculo de reserva de stock, validación de SKUs escaneados, generación de guía. En la app: widgets y lógica de estado (BLoC/Provider).	pytest (servicios Python) / JUnit-Jest según stack; flutter test para PickApp; Jasmine/Jest para Angular	Pedidos sin stock que avanzan (regla "sólo READY_FOR_ASSIGNMENT"), asignación por rol no autorizado, cálculos erróneos. Feedback en segundos.
Integration Tests	Un servicio con sus dependencias reales o emuladas : API ↔ Cloud SQL (transacciones, bloqueos de fila), API → Pub/Sub (mensaje publicado y consumido), Cloud Function ↔ Firestore.	pytest + Testcontainers (PostgreSQL), emulador de Pub/Sub y de Firestore (gcloud emulators), TestClient	Asignaciones duplicadas (concurrency contra BD real con SELECT ... FOR UPDATE /constraint UNIQUE), mensajes perdidos o duplicados en Pub/Sub (idempotencia del consumidor), dashboards desactualizados (Firestore no actualizado tras el evento).
Contract Tests	El formato acordado entre productor y consumidor: PickApp/Angular ↔ orders-api (OpenAPI), orders-api → Pub/Sub (esquema del evento), orders-api ↔ e-commerce externo.	Pact (consumer-driven) + Pact Broker; JSON Schema / Pub/Sub Schemas para eventos	Cambios de API que rompen la app mobile ya instalada en los dispositivos, eventos con campos renombrados que inventory-service no entiende.
End-to-End Tests	Flujos críticos completos en un ambiente integrado (staging): pedido → reserva → asignación → escaneo → guía → notificación; dashboard en tiempo real.	Playwright (Centro de Control), Appium (PickApp), pytest + API para preparar datos y verificar backend	Integración real entre los 5 microservicios, errores intermitentes al escanear , dashboard que no refleja el estado (verificación con polling sobre la UI).
UAT	Que la solución sirve al trabajo real: supervisores y operadores ejecutan escenarios de negocio en staging con dispositivos y scanners físicos reales.	Guiones de UAT en TestRail/Xray, sesiones exploratorias, checklist de aceptación	Usabilidad en almacén (guantes, luz, velocidad de escaneo), reglas de negocio mal interpretadas, hardware real (impresoras, scanners).

¿Qué se ejecuta en cada etapa del pipeline?

Etapa	Pruebas	Tiempo objetivo	Justificación (tiempo / costo)
Pull Request	Lint, SAST (Bandit/Semgrep), SCA (pip-audit/Dependabot), unit , contract (verificación del proveedor contra los pacts), integración ligera con emuladores, widget tests de Flutter; coverage gate sobre código nuevo	≤ 10 min	Se ejecuta decenas de veces al día: debe ser rápido y barato (runners estándar, sin ambientes). Corregir aquí cuesta minutos.
Merge a main	Todo lo anterior + integración completa (Testcontainers + emuladores), despliegue a dev/staging , API + seguridad + UI web (smoke y regresión crítica), k6 smoke, DAST baseline (ZAP)	≤ 25 min	Valida la integración real una vez por cambio aceptado. Requiere un ambiente efímero o compartido (costo moderado).
Release Candidate	Regresión E2E completa web + mobile en matriz de dispositivos , DAST completo, pruebas de rendimiento (carga +40%, spike), contratos con e-commerce, UAT con usuarios	2-4 h + UAT (1-2 días)	Es la última barrera antes del cliente. Lo más costoso (granja de dispositivos, ambientes tipo producción, horas de personas) se concentra aquí, sólo una vez por release.
Producción	Smoke post-deploy, canary con análisis automático de métricas, monitoreo sintético (journeys cada 5 min), alertas SLO	5-15 min por despliegue + continuo	Detecta lo que sólo ocurre con datos y tráfico reales. Costo bajo y continuo; reduce MTTR.

Costo relativo (estimado): unit $\approx 1x$ · integración $\approx 5x$ · contrato $\approx 2x$ · API $\approx 5x$ · E2E web $\approx 20x$ · E2E mobile $\approx 40x$ (dispositivos, mantenimiento de locators) · UAT $\approx 100x$ (horas de usuarios). Por eso la distribución objetivo es $\sim 70\%$ unit, $\sim 20\%$ integración/contrato/API y $\sim 10\%$ E2E.

1.2 Factibilidad de automatización de PickApp (Flutter)

Premisas técnicas: (1) Flutter dibuja su propia UI; la automatización necesita **Semantics identifiers** (o el driver de Flutter); (2) el **scanner físico** funciona en modo *keyboard wedge* o por *intent* (Zebra DataWedge), y ambos se pueden **simular** en un emulador, pero la óptica real no; (3) las impresoras requieren hardware.

#	Módulo	¿Automatizable?	Complejidad	Limitaciones técnicas	Prioridad
1	Pedidos Asignados	Sí	Baja	Requiere datos de prueba sembrados por API y sincronización con Firestore (esperas por condición, no <code>sleep</code>).	Alta (P1): puerta de entrada al flujo crítico
2	Preparación de Pedido	Parcial	Alta	El escaneo se simula (inyección de texto + ENTER o broadcast de intent); no se puede automatizar la óptica (códigos dañados, reflejos, distancia) ni la ergonomía. Es donde ocurren los "errores intermitentes al escanear": la lógica sí se automatiza.	Alta (P1): flujo de mayor valor y mayor tasa de defectos
3	Despacho	Parcial	Media	La generación de guía se valida por API/UI; la impresión física de la etiqueta no (se valida con impresora virtual/mock del SDK).	Alta (P1)
4	Gestión de Usuarios	Sí	Baja	Formularios CRUD estándar; mejor cubrir la mayor parte por API y dejar 1-2 casos de UI.	Media (P2)
5	Gestión de Insumos	Sí	Baja	CRUD; bajo riesgo de negocio.	Baja (P3)

#	Módulo	¿Automatizable?	Complejidad	Limitaciones técnicas	Prioridad
6	Dashboards	Parcial	Media	Los datos se validan por API; la parte visual (gráficos dibujados en canvas) se valida con aserciones sobre valores/semantics o <i>golden tests</i> de Flutter, no con Appium.	Media (P2)
7	Alertas y Notificaciones	Parcial	Alta	Las push (FCM) dependen del SO y de servicios externos: se valida la generación del evento por API y la recepción en emulador con FCM de prueba; la presentación nativa varía por dispositivo.	Media (P2)
8	Login / Autenticación	Sí	Baja	Si usa SSO corporativo con MFA, se usan usuarios de prueba sin MFA o <i>token injection</i> en ambientes de prueba.	Alta (P1): bloquea todo lo demás
9	Configuración de Impresoras	No (E2E) / Parcial (lógica)	Alta	Bluetooth/Wi-Fi con hardware real; el emulador no tiene Bluetooth. Se automatiza sólo la lógica con un <i>mock</i> del SDK de la impresora; el resto es manual en dispositivo físico.	Baja (P3): cambia poco, se prueba manualmente por release

Justificación. Se prioriza por **riesgo x frecuencia de uso x estabilidad**: los módulos 1, 2, 3 y 8 forman el flujo de ingresos diario y concentran los defectos reportados. Que la app esté en Flutter no impide automatizar, pero **exige acordar con desarrollo** que cada widget relevante tenga `Semantics(identifier: ...)` (equivalente al `data-testid` web), y complementar con `integration_test` de Flutter para lógica de UI. El hardware (scanner, impresora) se aísla detrás de interfaces que se pueden simular en pruebas y se valida físicamente en un conjunto pequeño de pruebas manuales por release.

1.3 Métricas de calidad

Métrica	Fórmula	Meta / umbral	Frecuencia
Defect Leakage	defectos encontrados en producción ÷ (defectos en pre-producción + en producción) × 100	< 5% (alerta si > 10%)	Por release y tendencia mensual
Test Coverage	(a) Código: líneas/ramas ejecutadas ÷ totales × 100. (b) Requisitos: requisitos con ≥1 prueba ÷ requisitos totales × 100	Código ≥ 80% (≥ 90% en código nuevo); regresión automatizada ≥ 70% de flujos críticos	Cada PR (código); quincenal (requisitos)
Change Failure Rate (DORA)	despliegues que causan incidente, rollback o hotfix ÷ despliegues totales × 100	< 15% (élite < 5%)	Semanal, tendencia mensual
MTTR (DORA)	Σ (hora de restauración – hora de detección) ÷ número de incidentes	< 1 hora para P1; < 4 h para P2	Por incidente; reporte mensual
Tasa de flakiness	ejecuciones con resultado distinto sin cambio de código ÷ ejecuciones totales × 100 (por test y por suite)	< 2% por suite; test con > 5% va a cuarentena	Diaria (job nocturno)
Pass rate del pipeline / tiempo de feedback	ejecuciones de PR en verde ÷ totales; mediana de duración del pipeline de PR	≥ 90% verdes; ≤ 10 min en PR	Semanal
Densidad de defectos por módulo (adicional)	defectos ÷ KLOC o ÷ puntos de historia, por módulo	Tendencia decreciente; foco en el top 3	Por sprint

Métrica	Fórmula	Meta / umbral	Frecuencia
Disponibilidad / SLO de la API (adicional)	requests exitosos (no 5xx) ÷ totales; p95 de latencia	99,9% mensual; p95 < 500 ms en /orders/assign	Continuo (Cloud Monitoring)

Parte 2. Diseño de casos de prueba funcionales

2.1 Matriz de casos de prueba: módulo Preparación de Pedido (PickApp)

Datos de referencia: operador 0P-312 activo; pedido de domicilio ORD-2025-007841 (2 SKUs); pedido de tienda ORD-2025-007842 (1 SKU); máximo configurado = 50 SKUs por pedido. Columna **Auto**:  candidato a automatizar ·  parcial ·  manual.

ID	Módulo	Caso de prueba	Precondiciones	Pasos	Resultado esperado	Tipo	Prioridad	Auto
TC-01	Preparación	Envío a domicilio completo genera guía	Operador activo; pedido HOME con 2 SKUs asignado	1. Login 2. Abrir pedido 3. Escanear SKU-1001 4. Escanear SKU-2002 5. Confirmar preparación	Avance 2/2; se genera número de guía; estado "Guía Generada" ; evento publicado a shipping-service	E2E	Crítica	 flujo de mayor valor, se repite en cada release
TC-02	Preparación	Recogida en tienda termina en "Listo para recoger"	Pedido PICKUP con 1 SKU asignado	1. Login 2. Abrir pedido 3. Escanear SKU 4. Confirmar 5. Marcar "Listo para recoger"	Estado "Listo para recoger" ; no se genera guía de envío	E2E	Crítica	
TC-03	Preparación	Rechazo por producto no disponible	Pedido asignado; un SKU sin stock físico	1. Abrir pedido 2. Tocar "Rechazar" 3. Elegir motivo "Producto no disponible" 4. Confirmar	Estado "Pendiente de resurtido" ; e-commerce notificado con el motivo; pedido sale de la lista del operador	E2E	Crítica	 (verificación del e-commerce vía API/mock)
TC-04	Login	Credenciales inválidas	App instalada	1. Usuario válido + contraseña incorrecta 2. Ingresar	Mensaje "Credenciales inválidas"; no se navega; no se guarda token; intento registrado	Negativo	Alta	
TC-05	Preparación	Scanner falla / lectura ilegible	Pedido abierto	1. Escanear un código dañado (lectura vacía o parcial)	Mensaje "Código vacío, vuelve a escanear"; avance sin cambios; el campo conserva el foco	Negativo	Alta	 la lógica sí (simulando lectura vacía); la óptica real es manual

ID	Módulo	Caso de prueba	Precondiciones	Pasos	Resultado esperado	Tipo	Prioridad	Auto
TC-06	Preparación	Producto escaneado no pertenece al pedido	Pedido abierto	1. Escanear EAN de otro producto	Mensaje "Producto X no pertenece al pedido"; avance sin cambios; botón Confirmar deshabilitado	Negativo	Alta	✓
TC-07	Preparación	SKU inexistente en catálogo	Pedido abierto	1. Digitar manualmente un código que no existe	Mensaje de producto no encontrado; se registra el evento para auditoría	Negativo	Media	✓
TC-08	Preparación	Pedido con un solo SKU	Pedido con 1 SKU	1. Abrir 2. Escanear el único SKU 3. Confirmar	Avance 1/1; Confirmar se habilita inmediatamente; flujo finaliza correctamente	Límite	Alta	✓
TC-09	Preparación	Pedido con el máximo de SKUs (50)	Pedido sembrado con 50 SKUs	1. Abrir 2. Escanear los 50 3. Confirmar	La lista hace scroll correctamente; avance 50/50; rendimiento aceptable (< 1 s por escaneo); guía generada	Límite	Media	✓ (datos sembrados por API)
TC-10	Preparación	Sesión expirada durante la preparación	Token con vida corta (ambiente de pruebas)	1. Abrir pedido 2. Escanear 1 SKU 3. Esperar expiración 4. Escanear el siguiente	Se pide re-autenticación sin perder el avance local; tras el login continúa en el mismo pedido	Límite	Alta	✓ (token de vida corta)
TC-11	Preparación	Escanear dos veces el mismo SKU	Pedido abierto con SKU ya escaneado	1. Escanear SKU-1001 2. Escanear SKU-1001 de nuevo	Mensaje "ya fue escaneado"; el contador no se duplica	Negativo	Alta	✓
TC-12	Preparación	Confirmar sin escanear todo	Pedido con 2 SKUs, 1 escaneado	1. Intentar Confirmar	Botón deshabilitado; no se puede despachar incompleto	Validación de datos	Alta	✓
TC-13	Preparación	Cantidades por SKU (qty > 1)	SKU con qty = 3	1. Escanear el SKU 3 veces	Avance por unidades (3/3); un 4.º escaneo se rechaza	Validación de datos	Media	✓

ID	Módulo	Caso de prueba	Precondiciones	Pasos	Resultado esperado	Tipo	Prioridad	Auto
TC-14	Rechazo	Rechazo sin motivo	Pedido abierto	1. Rechazar 2. Confirmar sin elegir motivo	Validación "Selecciona un motivo"; no cambia el estado	Validación de datos	Media	✓
TC-15	Preparación	Pérdida de red al confirmar	Pedido completo; modo avión durante la confirmación	1. Activar modo avión 2. Confirmar 3. Restaurar red	Mensaje de reintento; sin guías duplicadas al reconectar (idempotencia)	Negativo	Alta	⚠ (Appium puede cambiar la conectividad del emulador)
TC-16	Regresión	Lista de pedidos refleja reasignación del supervisor	Supervisor reasigna un pedido desde el Centro de Control	1. Operador en la lista 2. Supervisor reasigna 3. Refrescar	El pedido desaparece de la lista del operador original en < 5 s (tiempo real)	Regresión	Alta	✓ (API + mobile)
TC-17	Preparación	Usabilidad del scanner físico con guantes y poca luz	Dispositivo real + scanner Zebra	1. Escanear 10 productos en condiciones reales	Tasa de lectura ≥ 98%; sin lecturas fantasma	Funcional	Media	✗ hardware y ergonomía reales

Por qué son candidatos a automatización: se ejecutan en cada release (regresión), tienen resultado determinista, sus datos se pueden preparar por API y cubren el flujo de ingresos. TC-05 y TC-15 son parciales porque una parte depende de hardware o del sistema operativo; TC-17 es manual porque valida condiciones físicas.

Implementado en el repositorio: TC-01, TC-02, TC-03, TC-04 y TC-06 en `tests/mobile/test_pickapp_flow.py` (Appium) y `mobile-app/test/widget_test.dart` (Flutter).

2.2 Organización y gestión de los casos automatizables

¿Cómo los organizaría en suites? Con *markers* (etiquetas) combinables, no con carpetas duplicadas:

Suite	Contenido	Cuándo corre
smoke	TC-01, TC-04 (login + flujo feliz)	Cada despliegue (≈ 3 min)
critical	TC-01, TC-02, TC-03, TC-12	Cada PR/merge; si falla, bloquea
regression	Todos los ✓	Nocturno y en cada Release Candidate
scanner	TC-05, TC-06, TC-07, TC-11, TC-13	Cuando cambia el módulo de escaneo
boundary	TC-08, TC-09, TC-10	Nocturno
quarantine	Tests inestables en investigación	Job no bloqueante

Criterios para priorizar qué automatizar primero: 1. **Riesgo de negocio** (impacto × probabilidad): el flujo que genera ingresos y los defectos actuales. 2. **Frecuencia de ejecución:** lo que se repite en cada release devuelve antes la inversión. 3. **Estabilidad de la funcionalidad:** no automatizar pantallas que cambiarán el próximo sprint. 4. **Costo/factibilidad técnica:** primero lo determinista y con datos preparables por API. 5. **Nivel más bajo posible:** si una regla se puede probar por API, no se prueba por UI.

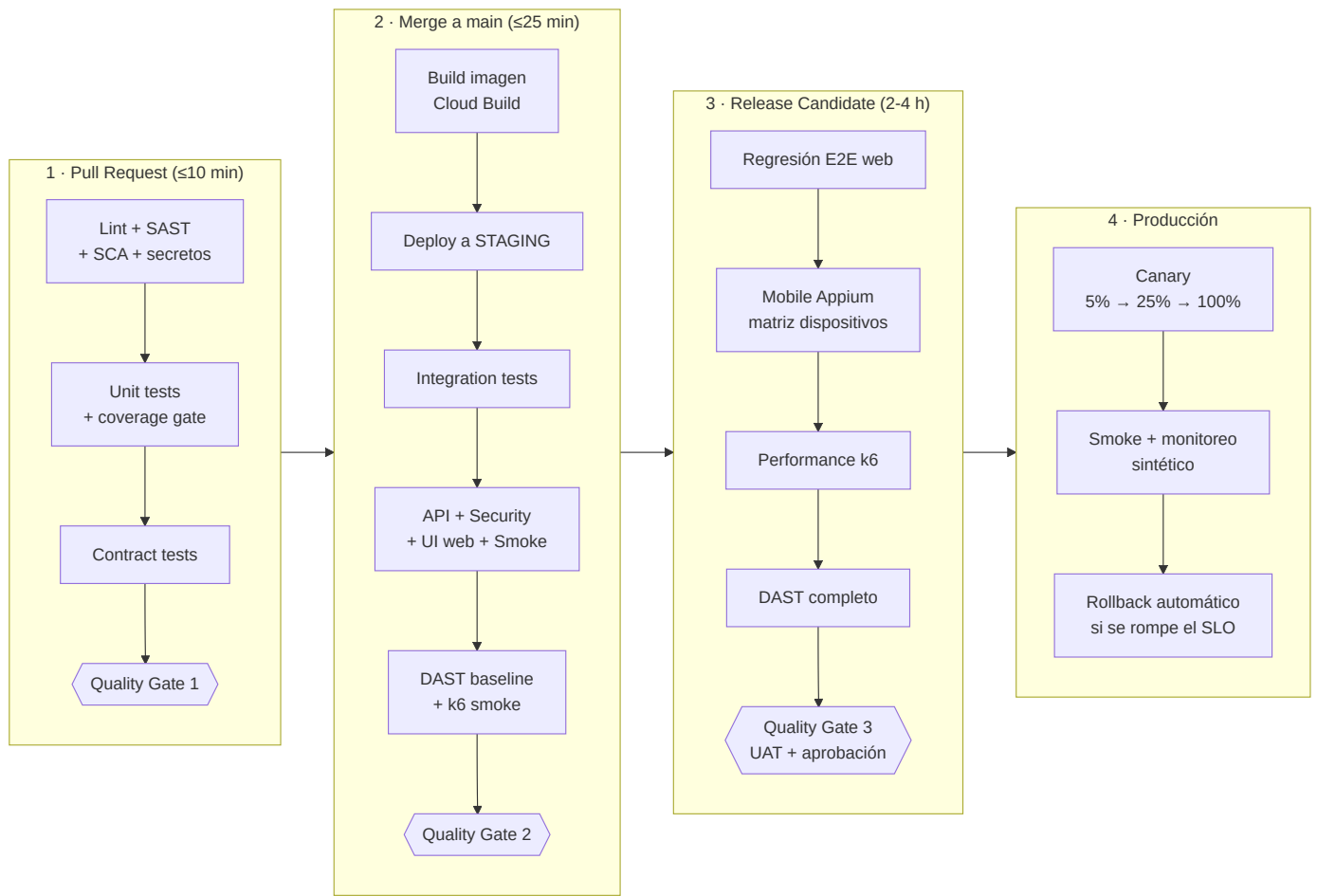
¿Cómo gestionaría los casos que requieren scanner físico? - Separar la **lógica** del **hardware**: la app recibe el código por una interfaz (`ScannerInput`); en pruebas se inyecta el código (texto + ENTER como un scanner *keyboard wedge*, o un *broadcast intent* como `DataWedge`). Implementado en `framework/mobile/barcode.py`. - Para cámara: imágenes de códigos en la *virtual scene* del emulador Android. - Un **conjunto pequeño de pruebas manuales en dispositivo físico** por release (TC-17), con una matriz de scanners/dispositivos reales usados en almacén. - Pruebas de **contrato con el hardware**: verificar los formatos (EAN-13, Code128, sufijo ENTER/TAB) que envía cada modelo de scanner.

¿Cómo integraría el feedback del equipo funcional? - Casos en una herramienta compartida (Xray/TestRail) con **trazabilidad** caso ↔ historia ↔ test automatizado. - **Revisión de casos en el refinamiento** (*three amigos*: negocio, desarrollo, QA) antes de automatizar, escritos en lenguaje de negocio (Gherkin opcional) para que el equipo funcional los valide. - Los defectos encontrados en pruebas manuales/UAT se convierten en **nuevos casos de regresión**. - Demo quincenal del reporte de automatización y del tablero de cobertura; el equipo funcional prioriza el backlog de automatización.

Parte 3. Automatización y CI/CD

Situación actual: Push a GitHub → Build → Deploy a Producción, sin validaciones intermedias.

3.1 Pipeline CI/CD ideal



Eta ­ pa	Qué se ejecuta	Tiempo estimado	Condi ­ ción de paso / blo ­ queo (Quality Gate)
Lint + SAST + SCA + secretos	Ruff/ESLint/flutter analyze, Semgrep/Bandit, pip-audit/Dependabot, Gitleaks	2-3 min	Bloquea con: error de lint, hallazgo SAST High/Critical, CVE High/Critical sin excepción aprobada, cualquier secreto en el código
Unit tests	pytest/JUnit/Jest + flutter test	2-4 min	100% pasan ; cobertura total ≥ 80% y ≥ 90% en código nuevo
Contract tests	Verificación de proveedor (Pact) + esquemas de eventos Pub/Sub	1-2 min	Bloquea si se rompe un contrato con un consumidor existente (can-i-deploy = no)
Build + firma	Imagen Docker en Cloud Build, escaneo de imagen (Artifact Analysis), firma (Binary Authorization)	3-5 min	Imagen sin vulnerabilidades críticas; firmada
Deploy a staging	Cloud Run (revisión nueva)	1-2 min	Health check /health OK
Integration tests	Servicios + Cloud SQL (Testcontainers), emuladores Pub/Sub/Firestore	4-6 min	100% pasan

Etapa	Qué se ejecuta	Tiempo estimado	Condición de paso / bloqueo (Quality Gate)
API + Security tests	<code>pytest -m "api or security"</code>	3-5 min	0 tests @critical fallidos y pass rate $\geq$ 95%
UI web + Smoke	Playwright (Centro de Control)	4-6 min	0 críticos fallidos; smoke 100%
DAST baseline + k6 smoke	ZAP baseline/API scan, k6 1 min	5 min	0 alertas ZAP en <i>FAIL</i> (inyección, XSS); k6: p95 < 500 ms, errores < 1%
Regresión E2E + Mobile	Playwright completo + Appium en 3-5 dispositivos	45-90 min	Pass rate $\geq$ 98%; 0 críticos; flaky en cuarentena no cuenta
Performance	k6 carga (+40%) y spike	30-60 min	p95 < 500 ms, p99 < 1 s, errores < 1% al 140% del pico
Aprobación	UAT + revisión de reporte por QA Lead	1-2 días	Firma de aprobación (environment protection rule en GitHub)
Canary + producción	Tráfico gradual en Cloud Run + smoke + sintéticos	30-60 min	Error 5xx < 1% y p95 < 800 ms durante cada escalón; si no → rollback automático

3.2 Mecanismos para impedir que código defectuoso llegue a producción

Branch protection rules (sobre `main` y `release/*`): - Prohibido el push directo; todo cambio entra por **Pull Request**. - $\geq$ 1 aprobación (2 en servicios críticos), con **CODEOWNERS** (QA revisa cambios en tests y contratos). - Las aprobaciones se descartan si llegan commits nuevos; conversaciones resueltas obligatorias. - Rama actualizada con `main` antes del merge; historial lineal; aplica también a administradores. - Commits firmados opcionales; prohibido force-push y borrado de la rama.

Required checks (deben estar en verde para habilitar "Merge"): `Lint + SAST + SCA`, `Unit + Integration + Contract (coverage gate)`, `API + Security + UI (Playwright)`, `DAST (OWASP ZAP)`, `Performance smoke (k6)`. El job de **cuarentena NO es requerido**.

Coverage gates: `--cov-fail-under=80` global y, en el PR, cobertura del **código modificado $\geq$ 90%** (diff-cover o Codecov "patch" status). La cobertura nunca puede **bajar** más de 1 punto respecto a `main`.

Canary deployments (Cloud Run): se despliega una revisión nueva **sin tráfico** (`--no-traffic`), se le asigna 5% (`gcloud run services update-traffic --to-revisions=NEW=5`), se observan 10-15 minutos las métricas de la revisión (5xx, p95, errores de Pub/Sub, métricas de negocio como "asignaciones/min"), y se avanza 25% → 50% → 100%. Para la app mobile: **staged rollout** en Play Store/TestFlight (1% → 10% → 100%) + **feature flags** (Firebase Remote Config).

Rollback automático: un job (Cloud Deploy con *verification* o un workflow) consulta Cloud Monitoring en cada escalón; si se cumplen las condiciones de rollback (5xx > 1%, p95 > 800 ms, caída de "pedidos asignados/min" > 20%), se ejecuta `gcloud run services update-traffic --to-revisions=PREVIOUS=100`, se notifica a Slack y se abre un incidente. Las migraciones de base de datos deben ser **compatibles hacia atrás** (*expand/contract*) para que el rollback sea seguro.

3.3 Pipeline funcional (GitHub Actions)

Archivo completo y comentado: `.github/workflows/ci.yml` (en el repositorio). Cumple cada requisito así:

Requisito	Cómo se cumple
Ejecute pruebas automatizadas	Jobs <code>unit-integration</code> , <code>functional-tests</code> (API + seguridad + UI), <code>performance-smoke</code> , <code>dast-zap</code>
Genere un reporte de resultados	<code>--junitxml + --html</code> (pytest-html), resumen de k6, reporte HTML de ZAP, resumen Markdown en la página de la ejecución (<code>\$GITHUB_STEP_SUMMARY</code>)
Publique el reporte como artefacto	<code>actions/upload-artifact@v4</code> con <code>if: always()</code> (se publica aunque fallen las pruebas)
Falle si hay tests críticos en rojo	<code>tools/quality_gate.py</code> lee el JUnit; los tests <code>@pytest.mark.critical</code> llevan la propiedad <code>critical=true</code> ; si alguno falló → <code>exit 1</code>
Notificación del resultado	Job <code>notify</code> (<code>if: always()</code>) envía a Slack vía webhook (secreto <code>SLACK_WEBHOOK_URL</code>) con el estado de cada job y el enlace a los reportes

Fragmento clave (quality gate):

```
- name: Ejecutar pruebas funcionales
  id: tests
  continue-on-error: true      # dejamos que el QUALITY GATE decida
  run: >
    pytest -m "(api or security or ui) and not quarantine"
    --reruns 1 --reruns-delay 1
    --junitxml=reports/junit-functional.xml
    --html=reports/functional-report.html --self-contained-html

- name: Publicar reportes como artefacto
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: functional-test-reports
    path: reports/

- name: QUALITY GATE - bloquear si hay tests críticos en rojo
  run: python tools/quality_gate.py "reports/junit-functional.xml"
```

Evidencia local del gate bloqueando (se provocó el bug de concurrencia con `ASSIGNMENT_LOCK_ENABLED=false`):

```
## Quality Gate: ❌ BLOQUEADO
| Tests ejecutados | 40 | Pasaron | 38 | Fallaron | 2 | Críticos fallidos | 1 |
### Tests críticos en rojo
- tests.api.test_assign_concurrency::test_two_supervisors_assign_same_order_simultaneously
exit=1
```

Evidencia: ejecución real en GitHub Actions

Ejecución #2 del workflow `CI - Quality Pipeline` (commit `7e116d8`): <https://github.com/restrej/prueba-tecnica-cloud-automation-and-mobile/actions/runs/36812798479>. Resultado: **success**.

Job	Resultado	Duración
Lint + SAST + SCA (Ruff, Bandit, Semgrep, pip-audit)	✅ success	43 s
Unit + Integration + Contract (coverage gate)	✅ success	23 s
API + Security + UI (Playwright) + Quality Gate	✅ success	53 s

Job	Resultado	Duración
Performance smoke (k6)	✓ success	1 min 24 s
DAST (OWASP ZAP)	✓ success	2 min 57 s
Quarantine (no bloqueante) + reporte de flakiness	✓ success	45 s
Notificar resultado	✓ success (paso de Slack omitido: falta configurar el secreto SLACK_WEBHOOK_URL)	3 s

Duración total del pipeline: **≈ 4 minutos**, dentro del objetivo de 10 minutos para PR.

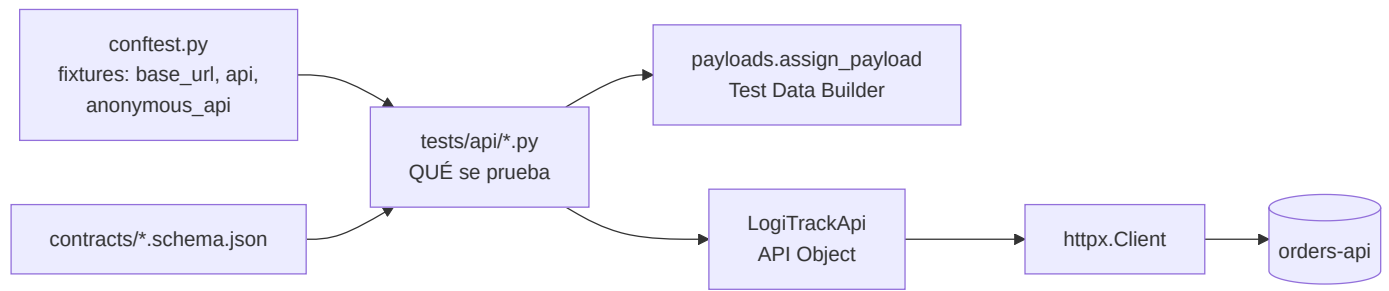
La ejecución #1 falló en el job estático porque `pip install semgrep` no encontró un paquete para el runner; se corrigió usando la imagen Docker oficial de Semgrep (commit `7e116d8`). Así funciona el pipeline: el problema se detectó y se corrigió antes de llegar a `main`.

Parte 4. Automatización técnica

Ejercicio A: API testing de `POST /api/v1/orders/assign`

Herramienta y arquitectura

Python + pytest + httpx, con estos patrones:



- **API Object** (`framework/api/logitrack_api.py`): un método por operación (`login` , `assign_order`); los tests no arman URLs ni headers.
- **Test Data Builder** (`assign_payload(**cambios)`): payload válido por defecto; cada test cambia **sólo** el campo que prueba.
- **Fixtures** (`tests/conftest.py`): levantan el servicio, entregan clientes autenticados y aíslan datos (cada test crea **su propio pedido** con `create_test_order()` → sin dependencias entre tests, paralelizable).
- **Markers** (`api` , `critical` , `smoke` , `concurrency`) para armar suites y para el quality gate.
- **Validación de contrato** con JSON Schema además de las aserciones funcionales.

Diseño de casos (implementados: 40 casos parametrizados)

Categoría	Caso	Esperado	Archivo
Positivo	Asignación con <code>NORMAL</code> , <code>URGENT</code> , <code>EXPRESS</code>	201 + cuerpo + evento Pub/Sub	<code>test_assign_positive.py</code>
Positivo	Reintento idéntico (misma orden y operador)	200, mismo <code>assignmentId</code> (idempotente)	<code>test_assign_positive.py</code>
Negativo	Token vacío, texto libre, JWT basura, firma falsa	401 + <code>WWW-Authenticate: Bearer</code>	<code>test_assign_negative.py</code>
Negativo	Pedido inexistente	404 <code>ORDER_NOT_FOUND</code>	"
Negativo	Operador inactivo (<code>OP-999</code>)	422 <code>OPERATOR_INACTIVE</code>	"
Negativo	Almacén inexistente / formato inválido / no corresponde	404 / 400 / 422	"
Negativo	Pedido sin stock (<code>PENDING_RESTOCK</code>)	409 <code>ORDER_NOT_ASSIGNABLE</code>	"
Límite	Cada campo vacío <code>" "</code> y cada campo ausente	400 indicando el campo	<code>test_assign_boundary.py</code>
Límite	IDs mal formados (minúsculas, dígitos de más/menos, espacios, tipo numérico, enum inválido)	400	"
Límite	<code>operatorId</code> con 1 y 6 dígitos (bordes válidos)	pasa validación (404 porque no existe)	"

Categoría	Caso	Esperado	Archivo
Límite	Pedido ya asignado a otro operador	409 ORDER_ALREADY_ASSIGNED, dueño intacto	"
Concurrencia	2 supervisores asignan el mismo pedido a la vez (threading.Barrier)	exactamente [201, 409] y 1 solo evento	test_assign_concurrency.py
Concurrencia	6 peticiones simultáneas a 3 operadores	1 sola creación; el resto 409/200	"

Prueba de que la prueba de concurrencia funciona: con `ASSIGNMENT_LOCK_ENABLED=false` el servicio pierde el bloqueo por pedido y la prueba falla con `assert [201, 201] == [201, 409]` (el bug real de "asignaciones duplicadas").

Código: caso positivo

```
@pytest.mark.smoke
@pytest.mark.critical
@pytest.mark.parametrize("priority", ["NORMAL", "URGENT", "EXPRESS"])
def test_assign_order_successfully_with_each_priority(api, priority):
    order_id = api.create_test_order() # Arrange: dato propio
    response = api.assign_order(assign_payload(orderId=order_id, priority=priority)) # Act
    assert response.status_code == 201, response.text # Assert
    body = response.json()
    assert body["orderId"] == order_id
    assert body["operatorId"] == "OP-312"
    assert body["status"] == "ASSIGNED"
    assert body["assignmentId"].startswith("ASG-")
    events = [e for e in api.published_events() if e["data"]["orderId"] == order_id]
    assert len(events) == 1 and events[0]["data"]["priority"] == priority # efecto en Pub/Sub
```

Código: casos negativos

```
@pytest.mark.critical
@pytest.mark.parametrize("token", ["", "token-invalido", "a.b.c", "eyJhbGciOiJIUzI1NiJ9.e30.firmaFalsa"],
                          ids=["vacío", "texto-libre", "jwt-basura", "firma-falsa"])
def test_invalid_token_is_rejected_with_401(anonymous_api, token):
    order_id = anonymous_api.create_test_order()
    response = anonymous_api.assign_order(assign_payload(orderId=order_id), token=token)
    assert response.status_code == 401
    assert response.json()["error"]["code"] == "UNAUTHORIZED"
    assert response.headers["WWW-Authenticate"] == "Bearer"

def test_inactive_operator_returns_422(api):
    order_id = api.create_test_order()
    response = api.assign_order(assign_payload(orderId=order_id, operatorId="OP-999"))
    assert response.status_code == 422
    assert response.json()["error"]["code"] == "OPERATOR_INACTIVE"
```

Código: concurrencia

```
def test_two_supervisors_assign_same_order_simultaneously(base_url, api):
    order_id = api.create_test_order()
    sup1 = _login_as(base_url, "supervisor1", PASSWORD)
    sup2 = _login_as(base_url, "supervisor2", PASSWORD)
    barrier = threading.Barrier(2) # ambos hilos salen en el MISMO instante

    def assign(client, operator_id):
        barrier.wait()
        return client.assign_order(assign_payload(orderId=order_id, operatorId=operator_id))

    with ThreadPoolExecutor(max_workers=2) as pool:
        futures = [pool.submit(assign, sup1, "OP-312"), pool.submit(assign, sup2, "OP-313")]
        statuses = sorted(f.result().status_code for f in futures)
        assert statuses == [201, 409], f"Posible condición de carrera: {statuses}"
```

Ejercicio B: Web UI automation (login del Centro de Control)

Herramienta: Playwright (Python) + pytest-playwright. **Por qué:** espera automática antes de cada acción (menos *flakiness*), aserciones `expect` que reintentan, aislamiento por contexto de navegador, trazas/video/capturas en fallos y soporte de Chromium, Firefox y WebKit. **Patrón:** Page Object Model (`framework/ui/pages/`).

Suite (18 casos, `tests/ui/test_login.py`)

Grupo	Casos
Happy path	Login exitoso y verificación de landing (URL, título, bienvenida con el nombre, tabla con pedidos); login con Enter; logout
Credenciales inválidas	Usuario incorrecto, contraseña incorrecta, ambos incorrectos → mensaje genérico, sin token en <code>sessionStorage</code>
Validaciones de campos	Ambos vacíos, usuario vacío, contraseña vacía, sólo espacios; caracteres especiales (<code><script></code> , <code>' OR '1'='1</code> , espacios, ñandú#\$\$); longitud máxima (50 usuario / 64 contraseña); contraseña enmascarada
Navegación / seguridad	Enlace "¿Olvidaste tu contraseña?"; acceso directo al panel sin sesión redirige al login

Estrategia de locators (resiliencia ante cambios del DOM)

Estrategia	Estabilidad	Uso
data-testid	★★★★★ Atributo exclusivo para pruebas: no cambia con estilos, textos, traducciones ni reestructuración	Primera opción (<code>page.get_by_test_id("login-submit")</code>)
Rol + nombre accesible (<code>get_by_role</code>)	★★★★★ Refleja lo que ve el usuario y valida accesibilidad; cambia si cambia el texto	Enlaces y botones con texto estable
IDs	★★ En Angular Material son autogenerados (<code>mat-input-0</code>) y cambian según el orden de renderizado	Sólo IDs semánticos y estables
CSS selectors	★★ Acoplados a clases de estilo (<code>.btn-primary</code>) que el diseño cambia	Último recurso, cortos
XPath	★ Acoplado a la estructura (<code>/div[2]/form/button</code>): cualquier <code><div></code> nuevo lo rompe; difícil de leer	Evitar

La pantalla simulada (`sut/web/login.html`) imita a propósito IDs autogenerados de Angular (`mat-input-0`) para mostrar por qué no se usan. Se acuerda con desarrollo una **convención de data-testid** (`<pantalla>-<elemento>`) revisada en el code review, y una regla de lint que impida eliminarlos.

Código: happy path

```
class LoginPage:
    PATH = "/control/login"

    def __init__(self, page: Page) -> None:
        self.page = page
        self.username_input = page.get_by_test_id("login-username")
        self.password_input = page.get_by_test_id("login-password")
        self.submit_button = page.get_by_test_id("login-submit")

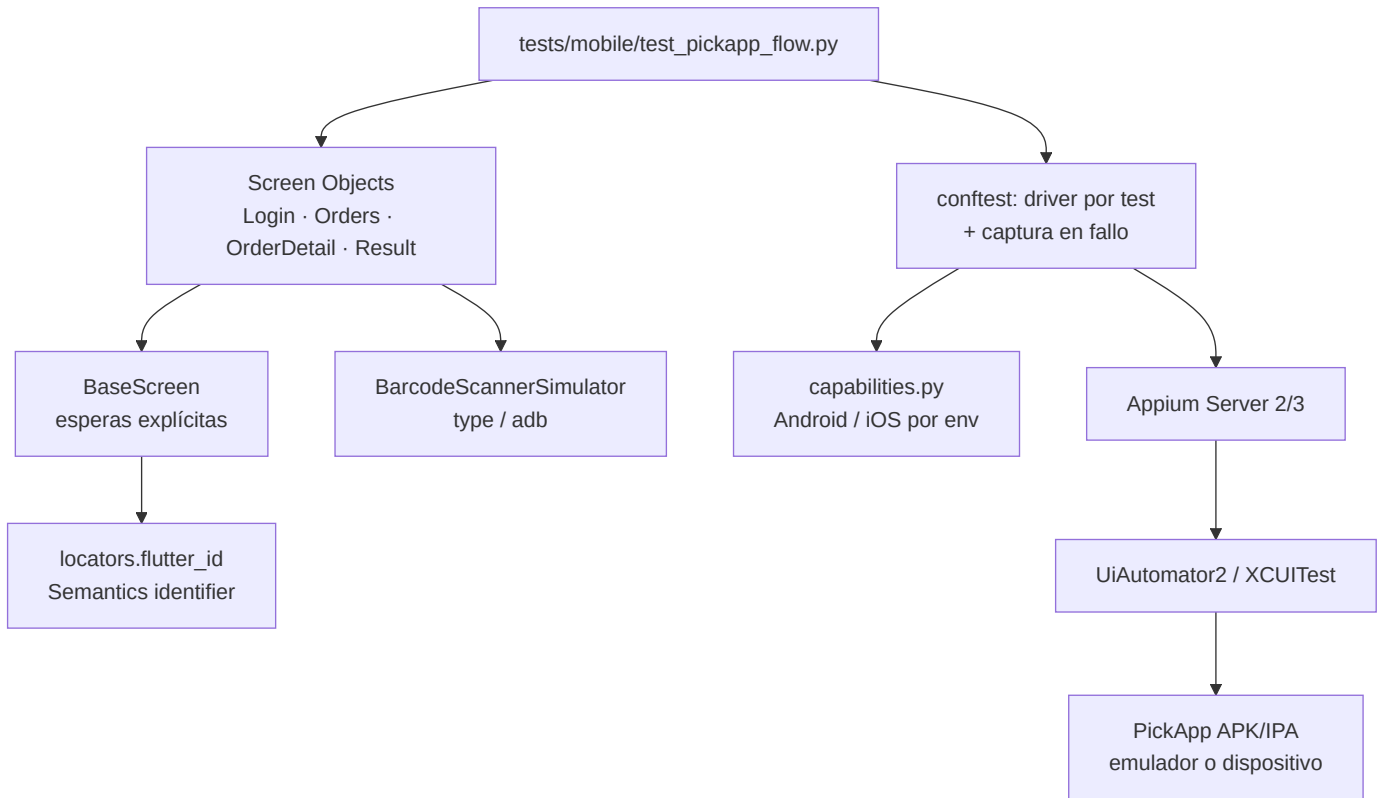
    def open(self) -> "LoginPage":
        self.page.goto(self.PATH)
        expect(self.page.get_by_test_id("login-title")).to_be_visible()
        return self

    def login(self, username: str, password: str) -> None:
        self.username_input.fill(username)
        self.password_input.fill(password)
        self.submit_button.click()

@pytest.mark.smoke
@pytest.mark.critical
def test_successful_login_shows_landing_page(page: Page):
    LoginPage(page).open().login("supervisor1", "Sup3rvisor!2025")
    dashboard = DashboardPage(page)
    expect(page).to_have_url(re.compile(r".*/control/dashboard$"))
    expect(dashboard.title).to_have_text("Panel de pedidos")
    expect(dashboard.welcome_message).to_have_text("Bienvenido, Laura Gómez")
    expect(dashboard.orders_table.locator("tbody tr").first).to_be_visible()
```

Ejercicio C: Mobile automation (PickApp en Flutter)

C.1 Arquitectura del framework mobile



¿Qué herramienta? Comparación:

Opción	Ventajas	Desventajas
Appium (UiAutomator2/XCUITest)	Caja negra sobre el binario real (igual que el usuario), Android + iOS, interactúa con el sistema (permisos, notificaciones, red, teclas), cualquier lenguaje (Python = mismo stack que API/UI), granjas de dispositivos (BrowserStack, Firebase Test Lab)	Requiere que Flutter exponga Semantics ; más lento que las pruebas nativas de Flutter
Flutter integration_test	Rápido, acceso directo a widgets por Key, mantenido por el equipo de Flutter	Sólo Dart, corre dentro de la app (no ve diálogos del sistema ni otras apps); el equipo QA debe conocer Dart
Appium Flutter Driver / Flutter Integration Driver	Locators por Key desde Appium	Requiere compilar la app con la extensión del driver (build distinto al de producción)
Maestro	YAML muy simple, tolerante a esperas	Menos flexible para lógica compleja, datos dinámicos e integración con API

Elección: Appium + Python para E2E (mismo lenguaje y reportes que el resto del framework, prueba el binario real e interactúa con el SO: escáner, permisos, red) + **flutter test/integration_test** del equipo de desarrollo para la lógica de widgets (base de la pirámide mobile).

Locators: Flutter vs nativo. - *Nativo*: cada vista tiene **resource-id** (Android) / **accessibilityIdentifier** (iOS) definidos por el desarrollador. - *Flutter*: la UI se dibuja en un canvas; Appium sólo ve el **árbol de accesibilidad**. Se envuelve cada widget con **Semantics(identifier: 'login-button')** (Flutter ≥ 3.19), que se publica como **resource-id** en Android y **accessibilityIdentifier** en iOS. Se busca con **UiSelector().resourceId("...")** / **accessibility id**. Evitar XPath y textos (cambian con traducciones).

Capabilities (framework/mobile/capabilities.py):

```
options = UiAutomator2Options()
options.platform_name = "Android"
options.automation_name = "UiAutomator2"
options.device_name = os.getenv("ANDROID_DEVICE_NAME", "Android Emulator")
options.platform_version = os.getenv("ANDROID_PLATFORM_VERSION") # opcional
options.app = os.getenv("APP_PATH", "mobile-app/build/app/outputs/flutter-apk/app-debug.apk")
options.app_package = "com.logitrack.pickapp"
options.app_activity = ".MainActivity"
options.auto_grant_permissions = True # evita popups de permisos (cámara)
options.no_reset = False # app limpia en cada sesión
options.new_command_timeout = 120
options.set_capability("appium:disableWindowAnimation", True) # menos flakiness
# iOS: XCUITestOptions con platform_name="iOS", automation_name="XCUITest", bundle_id, device_name
```

C.2 Suite del flujo

Caso	Flujo
Happy path (smoke, crítico)	Login → lista → ORD-2025-007841 → escanear 2 productos (avance 0/2 → 1/2 → 2/2) → Confirmar → "Guía Generada" + número de guía
Recogida en tienda	... → "Listo para recoger"
Rechazo	... → "Pendiente de resurtido"
Scanner lee código ajeno	error "no pertenece al pedido", avance sin cambios
Credenciales inválidas	mensaje de error

Simulación del scanner en emulador (framework/mobile/barcode.py): 1. *Keyboard wedge* (lo que hacen los scanners Zebra/Honeywell): escribir el código en el campo enfocado + `KEYCODE_ENTER` (66). 2. A nivel de sistema operativo: `mobile: shell → adb shell input text <código> + input keyevent 66` (Appium con `--allow-insecure=adb_shell`). 3. Scanners con *intent* (Zebra DataWedge): `adb shell am broadcast -a com.logitrack.SCAN --es data <código>`. 4. Cámara: imagen del código en la *virtual scene* del emulador Android.

Esperas y sincronización: nunca `sleep`. Espera implícita en 0 y **esperas explícitas** (`WebDriverWait` con `visibility_of_element_located` / `element_to_be_clickable`, *polling* 300 ms, timeout configurable con `MOBILE_WAIT_TIMEOUT`); esperas por **condición de negocio** (`wait_for_progress("2/2")`); animaciones desactivadas; datos preparados por API antes de abrir la app.

Diferentes resoluciones/dispositivos: matriz en CI (`.github/workflows/mobile.yml`: Pixel 6/Android 14 y Nexus 5/Android 11, pantalla pequeña) parametrizada por variables de entorno; locators independientes de coordenadas; `hide_keyboard()` antes de tocar botones (pantallas chicas); en RC, granja de dispositivos reales (Firebase Test Lab/BrowserStack) con los **modelos usados en almacén** (p. ej. Zebra TC52) y orientación vertical/horizontal.

C.3 Código: login + interacción con elementos Flutter

```

def flutter_id(identifier: str) -> tuple[str, str]:
    """Locator para un widget envuelto en Semantics(identifier: ...)."""
    if os.getenv("MOBILE_PLATFORM", "android").lower() == "ios":
        return AppiumBy.ACCESSIBILITY_ID, identifier
    return AppiumBy.ANDROID_UIAUTOMATOR, f'new UiSelector().resourceId("{identifier}")'

class LoginScreen(BaseScreen):
    def login(self, username: str, password: str) -> "OrdersScreen":
        self.type_text("login-username", username) # espera visible → click → clear → send_keys
        self.type_text("login-password", password)
        self.hide_keyboard()
        self.tap("login-button") # espera clickeable → click
        return OrdersScreen(self.driver)

def test_happy_path_preparation_generates_shipping_guide(driver):
    orders = LoginScreen(driver).login("OP-312", "Pick2025!")
    assert orders.is_loaded()
    detail = orders.open_order("ORD-2025-007841")
    detail.scan("7501234567890"); detail.wait_for_progress("1/2")
    detail.scan("7501234567891"); detail.wait_for_progress("2/2")
    result = detail.confirm()
    assert "Guía Generada" in result.status()

```

Del lado Flutter (mobile-app/lib/main.dart):

```

Widget testId(String id, Widget child) => Semantics(identifier: id, child: child);

testId('login-username', TextField(controller: _userController,
    decoration: const InputDecoration(labelText: 'Usuario'))),
testId('login-button', FilledButton(onPressed: _login, child: const Text('Ingresar'))),

```

Parte 5. Pruebas de rendimiento y seguridad

Ejercicio A: Rendimiento de `orders-api`

Cálculo de la carga: pico actual 500 req/min $\approx$ **8,3 req/s**; con +40% $\rightarrow$ **700 req/min $\approx$ 11,7 req/s**. Se diseña con margen: objetivo de carga = 12 req/s, estrés hasta 4x ($\approx$ 48 req/s).

5.1 Plan de pruebas

Tipo	Objetivo	Configuración (k6)	Métricas a medir	Criterios de aceptación
Carga	Confirmar que el pico del próximo año (700 req/min) se atiende de forma estable	<code>ramping-arrival-rate</code> : subida 5 min a 12 req/s, meseta 30 min , bajada 2 min; datos: pedidos únicos sembrados	p50/p95/p99, throughput, % error, instancias de Cloud Run, CPU/memoria, conexiones a Cloud SQL, latencia de Pub/Sub	p95 < 500 ms, p99 < 1 s, errores < 1% , sin crecimiento sostenido de latencia
Estrés	Encontrar el punto de quiebre y verificar recuperación	Escalones de 5 min: 12 $\rightarrow$ 24 $\rightarrow$ 36 $\rightarrow$ 48 req/s y vuelta a 0	Lo anterior + req/s máximos con p95 < 1 s, tipo de error al saturar (5xx, timeouts, 429), tiempo de recuperación	Degradación controlada (429/503 con mensaje, no 500 ni datos corruptos), p95 < 2 s, errores < 5%, recuperación < 2 min; documentar la capacidad máxima
Resistencia (Soak)	Detectar fugas de memoria, conexiones no liberadas, crecimiento de colas	9 req/s (pico actual) durante 4 horas	Memoria por instancia en el tiempo, pool de conexiones, <i>backlog</i> de Pub/Sub, p95 por hora, errores	p95 y memoria estables (variación < 10% entre la 1.ª y la 4.ª hora), errores < 1%, backlog de Pub/Sub sin crecer
Picos (Spike)	Simular la apertura simultánea de almacenes a las 7:00	2 req/s $\rightarrow$ 48 req/s en 10 s , sostener 3 min, bajar en 10 s, observar 3 min	Tiempo de <i>cold start</i> y de escalado de Cloud Run, errores durante el salto, p95	Errores < 2%, p95 < 1,5 s durante el pico, sin 5xx tras 60 s; ajustar <i>min-instances</i> si el <i>cold start</i> lo impide

Herramienta: k6. Justificación: scripts en JavaScript versionados junto al código y revisables en PR; **thresholds** que devuelven código de salida $\neq$ 0 (rompen el pipeline automáticamente); modelo de carga por **tasa de llegada** (exacto para "N req/min"); muy liviano (miles de VUs en un runner); salida a Prometheus/Grafana Cloud y a JSON. Frente a **JMeter** (GUI y XML difíciles de versionar/revisar), **Gatling** (Scala/Java, curva mayor) y **Locust** (Python, buena opción, pero menos eficiente por VU y sin thresholds nativos tan simples).

Implementación: `performance/k6/performance_plan.js` (-e `TEST_TYPE=load|stress|soak|spike`, -e `QUICK=1` para práctica).

5.2 Script de carga (50 VUs, 5 min, ramp-up 1 min)

`performance/k6/assign_load_test.js` (extracto):

```

export const options = {
  stages: [
    { duration: '1m', target: 50 }, // ramp-up de 0 a 50 usuarios virtuales
    { duration: '4m', target: 50 }, // 50 usuarios hasta completar 5 minutos
  ],
  summaryTrendStats: ['avg', 'min', 'med', 'p(90)', 'p(95)', 'p(99)', 'max'], // med = p50
  thresholds: {
    'http_req_duration{endpoint:assign}': ['p(95)<500', 'p(99)<1000'],
    http_req_failed: ['rate<0.01'], // tasa de error < 1%
    checks: ['rate>0.99'],
  },
};

export function setup() { // una vez: token + pedidos de prueba únicos
  return { token: login(), orderIds: seedOrders(20000) };
}

export default function (data) { // cada VU, en cada iteración
  const n = exec.scenario.iterationInTest;
  const res = assignOrder(data.token, data.orderIds[n % data.orderIds.length], n);
  assignErrors.add(res.status !== 201);
  sleep(1); // think time del supervisor
}

```

Resultado de la ejecución real (servicio simulado con 50 ms de latencia artificial de BD):

```

===== RESULTADO PRUEBA DE CARGA: POST /api/v1/orders/assign =====
Peticiones totales : 12826
Throughput         : 42.59 req/s (2555 req/min)
Tiempo resp. p50   : 53.5 ms
Tiempo resp. p95   : 57.2 ms
Tiempo resp. p99   : 60.9 ms
Tiempo resp. máx   : 111.2 ms
Tasa de error HTTP : 0.00 %
Asignaciones != 201: 0.00 %
VUs máximos        : 50
Umbral: OK p(95)<500 · OK p(99)<1000 · OK http_req_failed<1% · OK checks>99%

```

Nota: con 50 VUs y 1 s de *think time* se generan ≈ 2.555 req/min, **5 veces** el pico actual: la prueba del enunciado ya cubre holgadamente el +40%.

Ejercicio B: Seguridad

5.3 Pruebas de seguridad para `POST /api/v1/orders/assign`

Implementadas en `tests/security/test_assign_security.py` (45 casos) siguiendo OWASP API Security Top 10:

Área	Pruebas	Esperado
Autenticación	Sin token, token mal formado, firma falsificada, token expirado , alg: none, payload alterado (rol cambiado)	401 + WWW-Authenticate
Autorización / roles	Rol OPERATOR intenta asignar	403
Escalamiento horizontal	Supervisor de WH-01 asigna en WH-05 (BOLA/IDOR)	403

Área	Pruebas	Esperado
Escalamiento vertical	<i>Mass assignment</i> : enviar role, status, assignedBy extra	400 (campos extra prohibidos)
Enumeración de usuarios	Usuario inexistente vs contraseña incorrecta vs usuario bloqueado	Misma respuesta 401
SQL injection	' OR '1'='1, ; DROP TABLE, UNION SELECT en cada campo	400, nunca 500 ni 201; sin texto de BD en la respuesta
NoSQL injection	{"\$ne": null}, {"\$gt": ""}, {"\$where": ...}, arrays	400
Command / template injection	; ls -la /, && cat /etc/passwd, \${curl ...}, {{7*7}}	400
XSS	<script>, , javascript:, "><svg onload>	400 y el payload no se refleja ; Content-Type: application/json
Payloads malformados	JSON cortado, texto, vacío, null, array, número gigante, campo de 10.000 caracteres	400 sin <i>stack trace</i>
Content-Type	application/xml	415
Headers de seguridad	CSP (default-src 'self', frame-ancestors 'none'), X-Content-Type-Options: nosniff, X-Frame-Options: DENY, HSTS, Cache-Control: no-store, sin X-Powered-By	Presentes
CORS	Preflight desde el origen del Centro de Control vs https://evil.example	Sólo el origen confiable; nunca *
Rate limiting	Ráfaga de peticiones de un mismo usuario	429 + Retry-After

Complementos manuales (por release): pruebas de lógica de negocio con **Burp Suite** (manipular operatorId en tránsito, *replay* de peticiones, *race conditions* con *Turbo Intruder*), revisión de logs (que no contengan tokens).

5.4 Herramientas automatizadas y su lugar en el pipeline

Tipo	Herramienta	Etapas	Política
SAST	Semgrep (reglas OWASP Top 10), Bandit (Python), reglas S de Ruff; CodeQL como alternativa nativa de GitHub	Pull Request (≈ 1-2 min)	Bloquea con hallazgos <i>High</i> ; los demás generan comentario
Secretos	Gitleaks / GitHub secret scanning con <i>push protection</i>	Pre-commit + PR	Bloquea siempre
SCA	pip-audit , Dependabot, Trivy (imagen Docker)	PR + build de imagen	Bloquea CVE <i>Critical/High</i> con fix disponible
DAST	OWASP ZAP (API scan con OpenAPI y token de servicio)	Merge a main contra staging (baseline, ≈ 5 min) y Release Candidate (full scan)	Bloquea reglas marcadas FAIL (inyección, XSS, path traversal) en security/zap/zap-rules.tsv
Pentest manual	Burp Suite Professional	Antes de releases mayores / trimestral	Hallazgos críticos bloquean el release
Pruebas de seguridad propias	pytest (-m security)	PR / merge	Tests @critical bloquean

¿ZAP o Burp Suite? Burp Suite es el estándar de los pentesters para **pruebas manuales** (proxy, *Repeater, Intruder*); su escáner automático y la integración con CI requieren licencias de pago. **OWASP ZAP** es gratuito, tiene imágenes Docker y scripts listos para CI (`zap-api-scan.py`). Recomendación: **ZAP automatizado en el pipeline + Burp para exploración manual** por parte de QA/seguridad.

Resultado real del escaneo ZAP sobre la API simulada: **113 reglas PASS, 0 FAIL, 1 WARN** (*Unexpected Content-Type* en respuestas 404). Reporte: `reports/security/zap-report.html` (artefacto del job `dast-zap`).

Parte 6. Cloud y troubleshooting

6.1 Escenario 1: aumento de 500 tras el despliegue de `orders-api`

Hipótesis inicial: Cloud Run escala bien y los logs muestran *timeouts* hacia `inventory-service`; el despliegue probablemente cambió algo en la **llamada a inventory** (timeout, reintentos, pool de conexiones, nueva consulta) o `inventory` se degradó por el **aumento de carga** que genera la nueva versión (p. ej. reintentos agresivos que multiplican las peticiones).

Herramientas de GCP y orden de uso

#	Herramienta	Para qué	Qué busco
1	Cloud Monitoring	Confirmar el alcance y la correlación temporal	¿Los 5xx empiezan exactamente con la nueva revisión? Comparar por <code>revision_name</code> ; latencia p95/p99 de <code>orders-api</code> y de <code>inventory-service</code> ; CPU, memoria, instancias y concurrencia de <code>inventory</code> ; conexiones a Cloud SQL
2	Error Reporting	Ver qué errores son nuevos	Grupos de excepciones nuevos desde el despliegue (<code>TimeoutError</code> , <code>ConnectionPool exhausted</code>), versión donde aparecen por primera vez
3	Cloud Logging	Detalle y patrón de los fallos	Endpoints afectados, duración de los timeouts (¿siempre 5 s? = timeout configurado), ¿todas las instancias o algunas?, ¿algún almacén o pedido en particular?
4	Cloud Trace	Dónde se va el tiempo dentro de una petición	Spans de <code>orders-api</code> → <code>inventory-service</code> → Cloud SQL; ¿la latencia está en la red, en <code>inventory</code> o en su consulta a la BD? ¿hay N llamadas por petición (N+1)?

Paso previo: revisar el **diff del despliegue** (código y configuración: timeouts, variables de entorno, `max-instances`, `concurrency`, conector VPC, versión de librerías). Mitigación inmediata si el impacto es alto: **rollback** de tráfico a la revisión anterior (`gcloud run services update-traffic orders-api --to-revisions=PREV=100`) e investigar después.

Filtros de logs (Cloud Logging Query Language)

```
# 1) Errores 5xx de orders-api desde el despliegue, por revisión
resource.type="cloud_run_revision"
resource.labels.service_name="orders-api"
httpRequest.status>=500
timestamp>="2025-10-15T14:00:00Z"

# 2) Timeouts hacia inventory-service
resource.labels.service_name="orders-api"
severity>=WARNING
(textPayload:"inventory-service" OR jsonPayload.message:"inventory") AND
(textPayload:"timeout" OR jsonPayload.message:"timeout" OR jsonPayload.error:"DeadlineExceeded")

# 3) Lado inventory: peticiones lentas o rechazadas
resource.labels.service_name="inventory-service"
(httpRequest.latency>"3s" OR httpRequest.status>=500 OR httpRequest.status=429)

# 4) Todo lo de UNA petición en todos los servicios (correlación por trace)
trace="projects/logitrack-prod/traces/4bf92f3577b34da6a3ce929d0e0e4736"

# 5) Comparar revisiones
resource.labels.service_name="orders-api" httpRequest.status>=500
resource.labels.revision_name="orders-api-00042-xyz"
```

Útil además: **Log Analytics** (SQL sobre logs) para agrupar `COUNT(*)` de 5xx por `revision_name`, endpoint y minuto.

Correlación entre microservicios

- 1. **Trace context:** Cloud Run propaga `traceparent/X-Cloud-Trace-Context`; cada servicio lo reenvía en sus llamadas salientes y en los **atributos de los mensajes Pub/Sub**, y escribe en cada log el campo `logging.googleapis.com/trace` (el servicio simulado ya lo hace: `sut/main.py`). Así un `trace=...` muestra la petición completa en todos los servicios.
- 2. **Correlation/Request ID** de negocio (`X-Request-ID`, `orderId`) en todos los logs estructurados (JSON).
- 3. **Cloud Trace** para ver la cascada de spans y detectar el servicio lento.
- 4. **Alinear series de tiempo** en un mismo dashboard: 5xx de `orders-api` vs p99 de `inventory` vs conexiones de Cloud SQL vs `backlog` de `Pub/Sub`.
- 5. Confirmar la relación con la queja de `PickApp`: la app llama a `orders-api` → la lentitud de los operadores es el mismo timeout visto desde el cliente (Firebase Performance Monitoring / Crashlytics en la app).

Dashboards y alertas proactivas

Señal	Alerta (Cloud Monitoring)
Tasa de 5xx por servicio y revisión	> 1% durante 5 min → página al on-call
Latencia p95/p99 por servicio y endpoint	p95 > 500 ms (o > 2× la línea base) durante 10 min
SLO de disponibilidad (99,9%) con burn rate	Burn rate > 14× en 1 h (rápida) y > 6× en 6 h (lenta)
Errores/timeouts en llamadas salientes (métrica basada en logs: <code>inventory timeout</code>)	> 10/min
Instancias, CPU, memoria, concurrencia de Cloud Run	Instancias al <code>max-instances</code> > 5 min
Cloud SQL: conexiones activas, CPU, <code>lock waits</code>	Conexiones > 80% del máximo
Pub/Sub: <code>oldest_unacked_message_age</code> , <code>dead-letter</code>	Mensaje sin procesar > 5 min
Uptime checks / monitoreo sintético del flujo de asignación	2 fallos consecutivos
Métrica de negocio: asignaciones por minuto	Caída > 30% vs misma hora de la semana anterior
Comparación por revisión tras despliegue	Alimenta el rollback automático del canary

6.2 Escenario 2: E2E que falla 3 de cada 10 ejecuciones

Proceso paso a paso

- 1. **Recolectar evidencia sin cambiar nada:** ejecutar el test 30–50 veces (mismo commit, mismo ambiente) y guardar para cada ejecución: JUnit, logs de Appium, capturas/video, logs del backend por `trace/X-Request-ID`, hora y dispositivo. `tools/flaky_report.py` calcula la tasa de fallo.
- 2. **Clasificar los fallos:** ¿fallan siempre en el **mismo paso** y con el **mismo error**? ¿coinciden con horarios, dispositivos, `runners` o ejecución en paralelo?
- 3. **Reproducir aislando variables:** ejecutar solo (sin paralelismo), con otro dispositivo, contra un ambiente dedicado, con red estable vs red degradada.
- 4. **Aplicar el árbol de decisión** de abajo.
- 5. **Corregir la causa raíz,** ejecutar 50 veces seguidas en verde y documentar en el ticket. Mientras tanto, el test va a **cuarentena** (6.3).

Sospecha	Cómo lo confirmo	Cómo lo resuelvo
Test flaky (problema del test)	Patrones en el código: <code>sleep</code> fijos, esperas implícitas mezcladas con explícitas, locators por XPath/índice/texto, datos compartidos entre tests (mismo <code>orderId</code>), dependencia del orden de ejecución, verificación inmediata tras una operación asíncrona (Pub/Sub → Firestore), estado residual de la app (<code>noReset=true</code>)	Esperas explícitas por condición de negocio (polling hasta que la API devuelva <code>ASSIGNED</code>), datos únicos por test creados por API, <code>data-testid/Semantics</code> , app limpia por sesión, desactivar animaciones, test idempotente e independiente
Race condition (bug del sistema)	El backend muestra dos operaciones concurrentes sobre el mismo pedido (logs con el mismo <code>orderId</code> y timestamps cercanos); el fallo aumenta al ejecutar en paralelo o con latencia artificial; prueba dirigida con <code>threading.Barrier</code> (como <code>test_assign_concurrency.py</code>), y <code>ASSIGNMENT_LOCK_ENABLED=false</code> demuestra el efecto	Es un defecto del producto , no del test: bloqueo por fila / <code>UNIQUE</code> en BD, idempotency keys, consumidores Pub/Sub idempotentes, lecturas <i>read-your-writes</i> . El test se mantiene como detector
Infraestructura	Correlacionar los fallos con métricas: <i>cold starts</i> y escalado de Cloud Run, CPU/memoria del emulador o <i>runner</i> de CI, latencia y errores de red, conexiones de Cloud SQL, <i>backlog</i> de Pub/Sub, <i>oldest_unacked_message_age</i> , <i>throttling</i> (429) de servicios externos, salud de la granja de dispositivos	<code>min-instances</code> en el ambiente de pruebas, <i>runners</i> con KVM y recursos suficientes, ambiente de pruebas dedicado, <i>health check</i> previo a la suite, reintento sólo de pasos de infraestructura (instalar app, crear sesión)
Timeout mal configurado	Los fallos son <code>TimeoutException</code> justo en el límite (p. ej. siempre 10 s) y la operación termina poco después (el log del backend muestra éxito en 11-12 s); la distribución de tiempos (p95/p99) del paso supera el timeout	Timeouts basados en datos: p99 del paso + margen (p. ej. 1,5×); timeouts por tipo de operación (UI 10 s, propagación asíncrona 30 s con polling), configurables por ambiente (<code>MOBILE_WAIT_TIMEOUT</code>), alineados entre cliente HTTP, Appium (<code>newCommandTimeout</code>) y backend. Si el p99 real es inaceptable, es un defecto de rendimiento

6.3 Gestión de tests flaky (sin bloquear el pipeline y sin ignorarlos)

Estrategia implementada en el repositorio:

- Detección automática:** job nocturno que ejecuta la suite N veces y `tools/flaky_report.py`, que clasifica cada test: *estable*, **FLAKY** (falla a veces) o **ROTO** (falla siempre). También `pytest-rerunfailures` registra los tests que pasaron **sólo tras reintento** (son flaky aunque el pipeline esté verde).
- Política de reintentos acotada:** máximo **1 reintento** (`--reruns 1`) en el pipeline de PR/merge, siempre visible en el reporte. Nunca reintentos en tests unitarios. Un test que necesita reintento queda marcado.
- Cuarentena:** si la tasa de fallo es $> 5\%$ (o > 2 reintentos en una semana), se marca `@pytest.mark.quarantine`. El pipeline principal ejecuta `-m "not quarantine"`; el job `quarantine` lo sigue ejecutando con `continue-on-error: true` (**no bloquea** pero se ve en cada ejecución).
- Reglas para que no se ignoren:** cada test en cuarentena tiene **ticket, responsable y fecha límite** (SLA de 2 sprints); máximo **5% de la suite** en cuarentena (si se supera, se para a estabilizar); los tests **críticos** no pueden quedar en cuarentena más de 48 h sin cubrir su riesgo con otra prueba o una verificación manual.
- Reportería:** tablero semanal con tasa de flakiness por suite, top 10 tests inestables, tiempo medio en cuarentena y tendencia (meta $< 2\%$). Se publica en el resumen del pipeline (`$GITHUB_STEP_SUMMARY`) y en el informe de calidad.

6. **Salida de cuarentena:** tras corregir la causa raíz, 50 ejecuciones seguidas en verde → se quita el marker.

Evidencia (`tests/quarantine/test_flaky_demo.py` ejecutado 10 veces):

```
## Reporte de flakiness (10 ejecuciones)
| Test                                     | Pasó | Falló | Tasa de fallo | Clasificación
|
| tests.quarantine.test_flaky_demo::test_flaky_assignment... | 7    | 3    | 30%           | FLAKY ->
cuarentena + ticket |
```

Parte 7. Comunicación y presentación de resultados

Datos: 3 meses · pruebas manuales en 5 módulos · 40 casos automatizados · 23 defectos (3 críticos, 8 altos, 12 medios) · 35% de la regresión automatizada.

7.1 Estructura del informe ejecutivo de avance

1. Resumen ejecutivo (1 diapositiva). Semáforo general: ● **AMARILLO: no listo para producción.** Tres mensajes: (a) se encontraron y gestionaron 23 defectos antes de llegar al cliente, (b) quedan 3 críticos que bloquean el release, (c) la automatización cubre 35% de la regresión y apunta a 70% en 3 meses.

2. Alcance y avance.

Indicador	Valor	Visualización sugerida
Módulos probados manualmente	5 de 9 (56%)	Barra de progreso por módulo
Casos automatizados	40	Línea de tendencia semanal (acumulado)
Cobertura de regresión automatizada	35% (meta trimestral 70%)	Gauge / <i>bullet chart</i> con meta
Tiempo de regresión	manual ≈ 3 días → automatizado ≈ 25 min para el 35%	Barras comparativas

3. Estado de calidad del producto.

Severidad	Defectos encontrados	% del total
Crítico	3	13%
Alto	8	35%
Medio	12	52%
Total	23	100%

Visualizaciones: barras apiladas por severidad y estado (abierto/en curso/cerrado) y heatmap **módulo x severidad** para mostrar dónde se concentra el riesgo.

Más: tendencia de defectos abiertos vs cerrados (*burn-down*), defect leakage, tiempo medio de corrección por severidad.

¿Está listo para producción? No todavía. Criterios de salida propuestos: 0 críticos abiertos, ≤ 2 altos con *workaround* aprobado por negocio, 100% de los flujos críticos (pedido → asignación → escaneo → guía) en verde, regresión automatizada de flujos críticos en verde en 3 ejecuciones consecutivas, prueba de carga al +40% aprobada. **Qué falta:** corregir y re-probar los 3 críticos, probar los 4 módulos restantes (incluidos Despacho y Alertas), primera prueba de rendimiento y escaneo de seguridad.

4. Riesgos y mitigación.

Riesgo	Prob.	Impacto	Mitigación
3 defectos críticos sin corregir para la fecha	Media	Alto	Priorizar en el sprint actual; <i>daily</i> de seguimiento; re-test el mismo día

Riesgo	Prob.	Impacto	Mitigación
4 módulos sin probar	Alta	Alto	Plan de pruebas basado en riesgo: Despacho y Login primero
Sin pruebas de rendimiento ni seguridad	Alta	Alto	k6 y ZAP ya integrados en el pipeline (ver 3.3); ejecución completa en 2 semanas
Errores intermitentes del escáner	Media	Alto	Sesión en almacén con dispositivos reales + pruebas de simulación de escaneo
65% de la regresión aún manual	Alta	Medio	Roadmap de automatización priorizado por riesgo

5. Próximos pasos y estimaciones.

Acción	Esfuerzo	Fecha objetivo
Re-test de los 3 críticos + regresión	3 días	Semana 1
Pruebas manuales de los 4 módulos restantes	2 semanas	Semana 3
Automatizar 30 casos más (→ ~55% de regresión)	4 semanas	Semana 6
Baseline de rendimiento (carga + spike) y DAST completo	1 semana	Semana 2
Quality gates obligatorios en el pipeline	1 semana	Semana 2
Meta: 70% de regresión automatizada	3 meses	Mes 6

6. Decisiones que se piden al liderazgo: aprobar los criterios de salida, priorizar los críticos y asignar dispositivos/escáneres para el laboratorio de pruebas.

Formato: 6-8 diapositivas + anexo; dashboard vivo (Looker Studio/Grafana alimentado por Jira, GitHub Actions y Cloud Monitoring) para que el informe no sea una foto estática.

7.2 Respuesta al Gerente de Operaciones (≤ 200 palabras)

¿Por qué invertir en automatización si ya tenemos gente que prueba manualmente?

Porque el equipo manual no alcanza el ritmo del negocio. Publicamos cambios continuamente y hoy revisar a mano todo lo que ya funcionaba toma unos tres días; por eso se revisa solo una parte y los errores llegan a los almacenes.

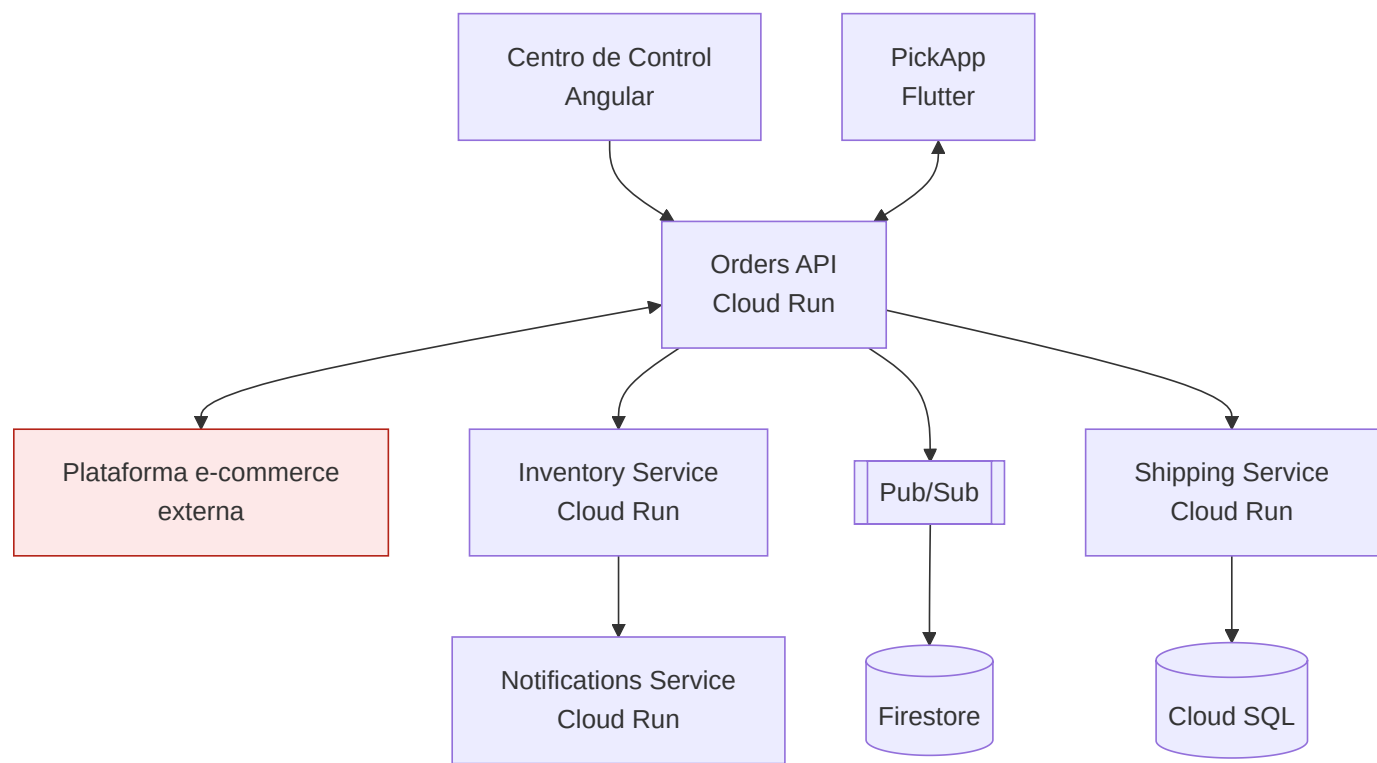
Un ejemplo de este proyecto: si un cambio vuelve a permitir que un pedido se asigne a dos operadores, hoy lo descubrimos cuando dos personas preparan el mismo pedido y otro cliente queda sin atender. Con automatización, esa verificación corre sola en minutos, en cada cambio y antes de llegar al almacén.

La automatización no reemplaza a las personas: hace las verificaciones repetitivas para que el equipo se concentre en lo que requiere criterio humano, como probar con escáneres reales en el almacén.

Retorno esperado: automatizar el 70% de la regresión libera alrededor de 15 días-persona por mes. Con una inversión inicial de unos tres meses, se recupera en aproximadamente seis meses, sin contar el ahorro más importante: menos pedidos retrasados, reprocesos y reclamos de clientes.

(167 palabras)

Bonus. Análisis de arquitectura y riesgos



B.1 Riesgos de calidad

Riesgo	Descripción	Prob.	Impacto	Prueba que lo detectaría	Mitigación
R1. Asignación duplicada por concurrencia	Dos supervisores (o reintentos de la app) asignan el mismo pedido; sin bloqueo transaccional ambos ganan	Alta	Alto	Prueba de concurrencia (<code>threading.Barrier</code>), integración contra Cloud SQL real, k6 con colisiones	Bloqueo por fila / <code>UNIQUE(order_id)</code> en asignaciones activas, idempotency key, respuesta 409
R2. Reserva de stock inconsistente	orders-api confirma el pedido antes de que inventory-service reserve; o el mensaje Pub/Sub se pierde/duplica (entrega <i>at-least-once</i>) → se procesan pedidos sin stock	Alta	Alto	Integración con emulador Pub/Sub (mensajes duplicados, desordenados, consumidor caído), contract test del evento, E2E pedido sin stock	Patrón Saga/outbox , consumidores idempotentes (dedupe por <code>eventId</code>), <i>dead-letter topic</i> + alertas, estado <code>PENDING_RESTOCK</code>
R3. Dependencia síncrona en cascada	orders-api llama síncronamente a inventory/shipping; si uno se degrada, orders-api agota hilos/conexiones y devuelve 500 (Escenario 1)	Media	Alto	Pruebas de resiliencia/caos (latencia inyectada, inventory caído), carga y estrés, tests de timeouts	Timeouts cortos + <i>retries</i> con <i>backoff</i> y <i>jitter</i> , circuit breaker , <i>bulkheads</i> , degradación elegante, mover a asíncrono donde se pueda

Riesgo	Descripción	Prob.	Impacto	Prueba que lo detectaría	Mitigación
R4. Ruptura de contratos	Cambio en la API rompe la app Flutter ya instalada (no todos actualizan), o el e-commerce externo cambia su API/webhook	Media	Alto	Contract testing (Pact), pruebas de compatibilidad con versiones N-1 de la app, monitoreo sintético del e-commerce	Versionado de API (/v1 , /v2), <i>can-i-deploy</i> en el pipeline, campos opcionales/aditivos, <i>tolerant reader</i>
R5. Estado en tiempo real desincronizado	Firestore (dashboards) se actualiza por eventos; si falla la Cloud Function o el evento llega tarde, el dashboard muestra datos viejos	Alta	Medio	E2E con verificación de propagación (latencia evento → Firestore < 5 s), integración con emulador de Firestore, monitoreo del <i>backlog</i>	Métrica y alerta de <i>lag</i> , reconciliación periódica Firestore ↔ Cloud SQL, indicador "última actualización" en la UI
R6. Seguridad del borde	Orders API expuesta a PickApp, Centro de Control y e-commerce: tokens, roles, IDOR entre almacenes, webhooks sin firma	Media	Alto	Suite de seguridad (authz por almacén, tokens), DAST (ZAP), pentest (Burp), validación de firma de webhooks	Gateway/IAP, OAuth2 con <i>scopes</i> por cliente, validación de HMAC en webhooks, Cloud Armor (rate limit/WAF)
R7. App mobile con conectividad intermitente	Almacenes con Wi-Fi inestable: escaneos perdidos o confirmaciones duplicadas	Alta	Medio	Pruebas mobile con red degradada (Appium <i>network_connection</i>), modo avión durante la confirmación	Cola local <i>offline-first</i> , reintentos idempotentes, sincronización con resolución de conflictos
R8. Datos y migraciones	Migraciones de Cloud SQL incompatibles con la versión anterior impiden el rollback	Baja	Alto	Pruebas de migración en staging con copia anonimizada, prueba de rollback	<i>Expand/contract</i> , migraciones versionadas y reversibles

B.2 Puntos de falla y estrategia de contract testing

Puntos de falla entre componentes: (1) PickApp ↔ Orders API (red móvil, versiones viejas de la app, auth); (2) Centro de Control ↔ Orders API; (3) Orders API ↔ e-commerce (sistema externo, SLA ajeno, cambios sin aviso, webhooks); (4) Orders API → Pub/Sub → consumidores (esquema, duplicados, orden, *dead letters*); (5) Orders API ↔ inventory/shipping (síncrono: latencia en cascada); (6) Pub/Sub → Cloud Functions → Firestore (lag); (7) servicios ↔ Cloud SQL.

Contrato	Tipo	Herramienta	Cómo
PickApp ↔ Orders API	HTTP, consumer-driven	Pact (<i>pact_dart</i> / <i>pact-python</i> o <i>Pact JS</i>) + Pact Broker	Las pruebas de la app (consumidor) generan el pact con las interacciones que usa (assign, listar pedidos, confirmar); orders-api (proveedor) lo verifica en su PR usando <i>provider states</i> ("pedido ORD-1 listo para asignar"). <i>can-i-deploy</i> impide desplegar un proveedor que rompa cualquier versión de la app aún en uso (se etiquetan las versiones de la app publicadas en tiendas).
Orders API ↔ e-commerce (externo)	HTTP con tercero que no ejecutará nuestras pruebas	Pact bidireccional (<i>bi-directional contract testing</i>) o Schemathesis contra la especificación OpenAPI del proveedor + mocks (WireMock) en integración	Se obtiene/pacta su OpenAPI; nuestro consumidor genera pacts que se comparan contra esa especificación ; además pruebas de contrato programadas contra su <i>sandbox</i> (detectan cambios no anunciados) y monitoreo sintético. Para los webhooks que nos envían: validar con JSON Schema y firma.

Contrato	Tipo	Herramienta	Cómo
Orders API ↔ Pub/Sub (eventos)	Mensajería asíncrona	Pact message contracts o JSON Schema/Avro registrado en Pub/Sub Schemas	Cada consumidor (inventory, notifications) publica sus expectativas del evento <code>order.assigned</code> ; el productor verifica que los mensajes que genera cumplen. Pub/Sub Schemas rechaza mensajes inválidos en el <i>topic</i> . Versionado del evento (<code>eventVersion</code>), cambios sólo aditivos; los incompatibles van a v2 con convivencia. Implementado: <code>contracts/order_assigned_event.schema.json</code> + <code>tests/contract/test_contracts.py</code> .

¿Por qué Pact? Es el estándar de *consumer-driven contracts*, soporta HTTP y mensajes, tiene librerías para Dart, JS, Python y JVM (stack heterogéneo: Flutter, Angular, Cloud Run), y el **Pact Broker** aporta la matriz de compatibilidad y `can-i-deploy` para el pipeline. **Spring Cloud Contract** sólo conviene si todo es JVM/Spring (*provider-driven*). **Schemathesis** no es contract testing dirigido por el consumidor, sino *property-based testing* de una especificación OpenAPI: excelente **complemento** para robustez del proveedor y para el sistema externo.

B.3 Plan de regresión E2E

Flujos críticos: en cada release (bloqueantes, ~30-45 min automatizados): 1. Pedido externo → reserva de stock → asignación → escaneo completo → guía generada (domicilio). 2. Recogida en tienda → "Listo para recoger". 3. Rechazo por falta de stock → "Pendiente de resurtido" + notificación al e-commerce. 4. Asignación concurrente / reasignación por supervisor (sin duplicados). 5. Login/autorización por rol y almacén (web + mobile). 6. Dashboard del supervisor refleja la asignación en < 5 s.

Semanales (o nocturnos, no bloqueantes del release salvo regresión): gestión de usuarios e insumos, reportes y dashboards de productividad, alertas y notificaciones (vencimientos/excepciones), casos límite (50 SKUs, sesión expirada, red intermitente), compatibilidad en la matriz completa de dispositivos y navegadores, impresoras (manual), soak de rendimiento y DAST completo.

Priorización de la automatización: puntaje = **impacto en el negocio** × **frecuencia de uso** × **probabilidad de fallo (historial de defectos)**, ajustado por estabilidad de la funcionalidad y costo de automatizar. Orden: (1) API de los flujos críticos (barata y estable), (2) contratos entre servicios, (3) UI web del supervisor, (4) mobile happy paths, (5) casos límite y negativos, (6) módulos de bajo riesgo. Cada defecto escapado a producción genera su test de regresión.

B.4 Estrategia integral para reducir defectos en producción

Palanca	Acciones concretas
Shift-left testing	QA en el refinamiento (<i>three amigos</i>) con criterios de aceptación verificables y ejemplos (BDD); <i>Definition of Done</i> con pruebas unitarias + contrato; pruebas de API automatizadas en el mismo PR de la funcionalidad; SAST/SCA y lint en <i>pre-commit</i> y PR; ambientes efímeros por PR en Cloud Run (revisión con <i>tag</i>) para probar antes del merge
Feature flags para releases graduales	Toda funcionalidad nueva detrás de un flag (Firebase Remote Config / LaunchDarkly / Unleash); activación por almacén piloto → región → todos; <i>kill switch</i> para apagar sin desplegar; probar ambos estados del flag en CI; limpieza de flags viejos (deuda técnica)
Monitoreo sintético en producción	Journeys automatizados cada 5 min (login, consultar pedidos, asignar un pedido de prueba marcado como sintético y revertirlo) desde varias regiones con Cloud Monitoring <i>synthetic monitors</i> (Playwright/Node); uptime checks por servicio; alertas por SLO y por métricas de negocio (asignaciones/min)

Palanca	Acciones concretas
Pruebas en ambientes pre-productivos	Staging con paridad con producción (misma infraestructura como código en Terraform, mismas versiones, datos anonimizados representativos); regresión E2E, rendimiento, DAST y UAT en RC; despliegue canary + rollback automático; <i>chaos testing</i> en staging (latencia en inventory, caída de Pub/Sub)
Code review con perspectiva de QA	Checklist en la plantilla de PR: ¿tiene tests al nivel correcto? ¿casos negativos y límites? ¿idempotencia y concurrencia en escrituras? ¿manejo de timeouts/reintentos? ¿logs estructurados con <code>trace</code> y sin datos sensibles? ¿cambios de contrato versionados? ¿ <code>data-testid</code> /Semantics en UI nueva? ¿migraciones reversibles? ¿flag para la funcionalidad?; CODEOWNERS incluye QA en <code>tests/</code> , <code>contracts/</code> y <code>workflows</code> ; <i>quality gates</i> como <i>required checks</i>
Aprendizaje continuo	Postmortem sin culpables de cada defecto escapado → test de regresión + acción preventiva; seguimiento mensual de defect leakage, change failure rate y MTTR

Anexo A. Implementación y cómo ejecutarla

Stack tecnológico

Necesidad	Herramienta	Por qué
Lenguaje y runner de pruebas	Python 3.11 + pytest	Sintaxis simple, fixtures, markers para armar suites, gran ecosistema
API testing	pytest + httpx + JSON Schema	Cliente HTTP simple; contratos validados con jsonschema
UI web	Playwright (Python)	Espera automática (menos flaky), locators por data-testid/rol, trazas y capturas
Mobile	Appium (UiAutomator2/XCUITest) + Python	Caja negra sobre el APK real, multiplataforma, mismo lenguaje que el resto
Rendimiento	k6	Scripts en JS versionables, umbrales que hacen fallar el pipeline, bajo consumo
Seguridad	pytest (OWASP) + Bandit/Semgrep (SAST) + pip-audit (SCA) + OWASP ZAP (DAST)	Gratuitas y automatizables en CI
CI/CD	GitHub Actions	Integrado con el repositorio, artefactos, required checks
Sistema bajo prueba	FastAPI (simulación de orders-api)	Permite ejecutar todo sin el backend real

Estructura del repositorio

```
├── sut/                                # Sistema bajo prueba SIMULADO: orders-api (FastAPI) + web de login
│   ├── main.py                        # rutas, middlewares (logs JSON, headers de seguridad), manejo de errores
│   ├── service.py                    # reglas de negocio de la asignación (incluye control de concurrencia)
│   ├── security.py                   # JWT, hash de contraseñas, rate limiting
│   ├── events.py                     # Pub/Sub simulado
│   └── web/                           # pantalla de login y dashboard del "Centro de Control"
├── framework/                         # Código reutilizable de automatización
│   ├── api/                          # API Object (LogiTrackApi) + Test Data Builder (payloads)
│   ├── ui/pages/                     # Page Objects (Playwright)
│   └── mobile/                       # capabilities, locators Flutter, simulador de scanner, Screen Objects
├── tests/
│   ├── unit/ integration/ contract/   # base de la pirámide
│   ├── api/                          # Ejercicio A: positivos, negativos, límite, concurrencia
│   ├── ui/                           # Ejercicio B: login del Centro de Control
│   ├── mobile/                       # Ejercicio C: flujo PickApp con Appium
│   ├── security/                     # 5.3: authn/authz, inyección, XSS, headers, CORS, rate limit
│   └── quarantine/                   # 6.3: demo de test flaky en cuarentena
├── contracts/                         # JSON Schemas: respuesta de la API, errores y evento Pub/Sub
├── performance/k6/                   # 5.1 plan (load/stress/soak/spike) y 5.2 script de carga
├── security/zap/                     # 5.4 DAST con OWASP ZAP
├── tools/                            # quality_gate.py y flaky_report.py
├── mobile-app/                       # PickApp demo en Flutter (objetivo de Appium) + widget tests
├── .github/workflows/                # ci.yml, mobile.yml, performance.yml
└── docs/                             # paso a paso, conceptos, respuestas y entregable PDF
```

Mapa: partes del PDF → implementación

Parte	Dónde está
1. Estrategia, factibilidad, métricas	docs/respuestas/ + docs/entregable/
2. Casos de prueba funcionales	docs/respuestas/ + docs/entregable/
3.1 / 3.2 Pipeline ideal y mecanismos	docs/respuestas/
3.3 Pipeline funcional	.github/workflows/ci.yml + tools/quality_gate.py
4-A API	tests/api/, framework/api/
4-B UI web	tests/ui/, framework/ui/pages/, sut/web/
4-C Mobile	tests/mobile/, framework/mobile/, mobile-app/, .github/workflows/mobile.yml
5.1 / 5.2 Rendimiento	performance/k6/
5.3 / 5.4 Seguridad	tests/security/, security/zap/, jobs SAST/DAST de ci.yml
6. Troubleshooting y flaky	docs/respuestas/, tests/quarantine/, tools/flaky_report.py
7. Comunicación	docs/respuestas/
Bonus	docs/respuestas/, contracts/, tests/contract/

Resultados de la ejecución local

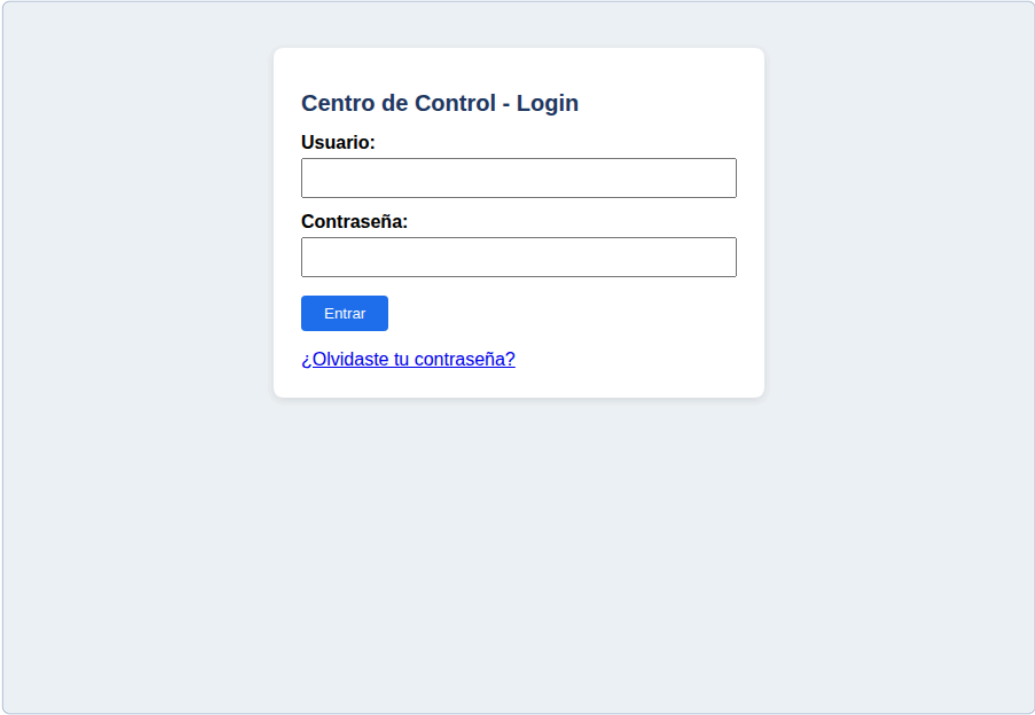
Suite	Resultado
Unit + Integration + Contract	45 pruebas OK · cobertura 96% (gate 80%)
Total sin emulador	148 pruebas OK · quality gate APROBADO
API (Ejercicio A)	40 OK, incluida la concurrencia (y falla como debe con ASSIGNMENT_LOCK_ENABLED=false)
Seguridad (5.3)	45 OK
UI Playwright (Ejercicio B)	18 OK
Mobile	7 pruebas unitarias del framework OK · 5 E2E de Appium listas (se saltan si no hay emulador) · 5 widget tests de Flutter OK
k6 5.2 (50 VUs, 5 min)	12.826 peticiones · 42,6 req/s · p50 53 ms · p95 57 ms · p99 61 ms · 0% errores
SAST (Bandit, Ruff, Semgrep)	0 hallazgos abiertos
DAST (OWASP ZAP API scan)	113 reglas OK · 0 FAIL · 1 WARN (Content-Type en rutas 404)
Flaky demo	7 pasa / 3 falla en 10 ejecuciones → clasificado FLAKY
GitHub Actions (ci.yml)	Ejecución #2 : 7/7 jobs en verde en ≈ 4 min

Limitaciones conocidas (transparencia)

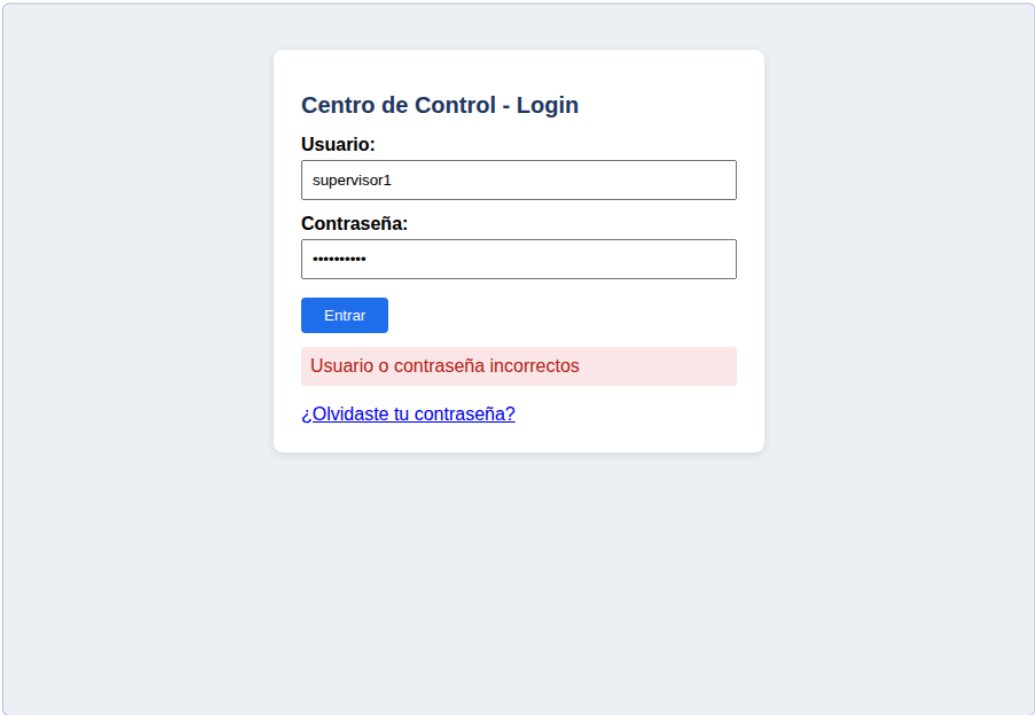
- El backend de LogiTrack es **simulado** (sut/): base de datos y Pub/Sub en memoria. Los tiempos de k6 incluyen una latencia artificial de 50 ms (SIMULATED_LATENCY_MS) para imitar una consulta a Cloud SQL.
- Los endpoints /api/v1/test-support/* existen sólo para preparar datos de prueba y se desactivan con ENABLE_TEST_SUPPORT=false (como debe ser en producción).
- Las pruebas E2E de Appium no se ejecutaron en el entorno de desarrollo de esta entrega (sin emulador Android disponible); la app compila (flutter analyze sin hallazgos) y sus widget tests pasan.

Se ejecutan con el paso 13 de la guía o con el workflow `mobile.yml`.

Anexo B. Evidencias de ejecución



Pantalla de login del Centro de Control (objetivo del Ejercicio B).



Credenciales inválidas: mensaje genérico (prueba automatizada).

Centro de Control - Login

Usuario:

El usuario contiene caracteres no permitidos

Contraseña:

La contraseña es obligatoria

Entrar

[¿Olvidaste tu contraseña?](#)

Validaciones de campos: caracteres especiales y contraseña vacía.

Panel de pedidos

Cerrar sesión

Bienvenido, Laura Gómez

Pedido	Almacén	Estado	Operador
ORD-2025-001001	WH-01	READY_FOR_ASSIGNMENT	-
ORD-2025-007841	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007842	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007843	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007844	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007845	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007846	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007847	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007848	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007849	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007850	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007851	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007852	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007853	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007854	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007855	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007856	WH-05	READY_FOR_ASSIGNMENT	-
ORD-2025-007857	WH-05	READY_FOR_ASSIGNMENT	-

Landing page tras login exitoso (verificada por el happy path).

reporte.html

Report generated on 01-Oct-2026 at 03:50:30 by [pytest.html](#) v4.2.0

Environment

Python	3.11.15
Platform	Linux-6.18.44-1c-v50-x86_64-with-glibc2.39
Packages	• pytest: 9.1.1 • pluggy: 1.6.0
Plugins	• html: 4.2.0 • metadata: 3.1.1 • playwright: 0.9.0 • rerunfailures: 16.7 • xdist: 3.8.0 • platformdirs: 4.12.2 • schemathesis: 4.28.0 • hypothesis: 6.168.3 • anyio: 4.15.1 • cov: 7.1.0 • base-url: 2.1.0
JAVA_HOME	/usr/lib/jvm/java-21-openjdk-amd64
Base URL	

Summary

148 tests took 00:00:23.

(Un)check the boxes to filter the results.

☒ 0 Failed, ☒ 148 Passed, ☐ 0 Skipped, ☐ 0 Expected failures, ☐ 0 Unexpected passes, ☐ 0 Errors, ☐ 0 Reruns ☐ 0 Retried,

[Show all details](#) / [Hide all details](#)

Result	Test	Duration	Links
Passed	tests/api/test_assign_boundary.py::test_empty_field_returns_400[orderId]	957 ms	
Passed	tests/api/test_assign_boundary.py::test_empty_field_returns_400[operatorId]	90 ms	
Passed	tests/api/test_assign_boundary.py::test_empty_field_returns_400[warehouseId]	84 ms	
Passed	tests/api/test_assign_boundary.py::test_empty_field_returns_400[priority]	84 ms	
Passed	tests/api/test_assign_boundary.py::test_missing_field_returns_400[orderId]	86 ms	
Passed	tests/api/test_assign_boundary.py::test_missing_field_returns_400[operatorId]	84 ms	
Passed	tests/api/test_assign_boundary.py::test_missing_field_returns_400[warehouseId]	89 ms	
Passed	tests/api/test_assign_boundary.py::test_missing_field_returns_400[priority]	85 ms	
Passed	tests/api/test_assign_boundary.py::test_invalid_id_formats_return_400[orderId-ORD-25-7841]	86 ms	
Passed	tests/api/test_assign_boundary.py::test_invalid_id_formats_return_400[orderId-ord-2025-007841]	84 ms	
Passed	tests/api/test_assign_boundary.py::test_invalid_id_formats_return_400[orderId-ORD-2025-0078410]	88 ms	
Passed	tests/api/test_assign_boundary.py::test_invalid_id_formats_return_400[orderId-ORD-2025-007841]	86 ms	
Passed	tests/api/test_assign_boundary.py::test_invalid_id_formats_return_400[operatorId-OP312]	84 ms	

Reporte HTML de pytest: 148 pruebas ejecutadas, 0 fallidas.



ZAP Scanning Report

Site: <http://localhost:8000>

Generated on Thu, 1 Oct 2026 03:48:59

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	1
Informational	3
False Positives	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP errors logged - see the zap.log file for details	3
Low	Warning		ZAP warnings logged - see the zap.log file for details	8
Low	Exceeded High	http://localhost:8000	Percentage of responses with status code 4xx	91 %
Info	Informational		Percentage of network failures	1 %
Info	Informational	http://localhost:8000	Percentage of responses with status code 2xx	5 %
Info	Informational	http://localhost:8000	Percentage of responses with status code 3xx	2 %
Info	Informational	http://localhost:8000	Percentage of endpoints with content type application/json	96 %
Info	Informational	http://localhost:8000	Percentage of endpoints with content type text/css	2 %
Info	Informational	http://localhost:8000	Percentage of endpoints with content type text/html	1 %
Info	Informational	http://localhost:8000	Percentage of endpoints with content type text/javascript	2 %
Info	Informational	http://localhost:8000	Percentage of endpoints with method GET	75 %
Info	Informational	http://localhost:8000	Percentage of endpoints with method POST	24 %
Info	Informational	http://localhost:8000	Count of total endpoints	151

Reporte de OWASP ZAP (DAST) sobre la API: 0 alertas FAIL.